Software Automation

- **Machine Learning (ML)**
    - **Definition**:
        - Branch of **Artificial Intelligence (AI)**.
        - Focuses on **developing algorithms and models**.
        - Enables computers to **learn from data**.
        - Makes **predictions or decisions without being explicitly programmed** for each task.
    - **How ML Works**:
        - **Foundation: Data** (numbers, text, images, audio).
            - More relevant data improves learning.
        - **Algorithms**: Mathematical procedures for learning from data.
            - Different algorithms suit different tasks.
        - **Training Phase**:
            - Machine learns from a **dataset**.
            - Finds **patterns and relationships**.
            - Uses information for predictions/decisions.
        - **Testing Phase**:
            - Model tested with **new data**.
            - Evaluates performance.
            - Checks generalisation to **unseen data**.
    - **Main Types of ML**:
        - **Supervised Learning**:
            - Trained on **labelled dataset** (input + correct output).
            - Learns to **predict output based on input**.
            - Example: Predicting house prices based on features.
        - **Unsupervised Learning**:
            - Trained on data **without labelled outputs**.
            - Tries to find **patterns or groupings** on its own.
            - Example: Clustering customers based on purchasing behaviour.
        - **Reinforcement Learning**:
            - Learns by **interacting with an environment**.
            - Receives **feedback (rewards/penalties)**.
            - Model updated based on feedback.
            - Repeated until **optimal decisions** are made.
            - Often used in game playing or robotics.

- **Applications of ML**:
  - **Image recognition**: Identifying objects, people, scenes (e.g., facial recognition).
  - **Natural Language Processing (NLP)**: Understanding/interpreting human language (e.g., chatbots, language translation).
  - **Recommendation systems**: Recommending movies/products based on user preferences.
  - **Fraud detection**: Analysing transactions for suspicious activity.
  - Online recommender systems.
  - Google search algorithms.
  - Facebook auto friend tagging suggestions.
- **Challenges in Using ML**:
  - **Quality of Data**: Success depends heavily on quality and quantity of training data.
  - **Overfitting**: Model learns training data too well, performing poorly on new data.
  - **Bias**: Biased or unrepresentative training data can perpetuate biases in predictions.
- **Variables in ML**:
  - **Definition**: A **characteristic, property, or feature that can change** and represents data.
  - **Types**:
    - **Features (Independent Input Variables)**: Information used by the model to **make predictions**.
      - Example: House size, number of bedrooms, location, age.
    - **Target Variable (Dependent Output Variable)**: The outcome the model predicts.
      - Example: Pass or Fail (for a test).
- **ML vs. Traditional Computing**:
  - **Traditional Computing**:
    - Purpose: **Decision support based on predefined rules**.
    - Implementation: **Static, rule-based systems**.
    - Adaptability: **Does not adapt** to new information.
    - Use cases: Business processes, troubleshooting.
  - **Machine Learning**:
    - Purpose: **Predictive modelling based on learned patterns**.
    - Implementation: **Dynamic, data-driven learning**.
    - Adaptability: **Learns and improves** with new data.
    - Use cases: Predictive analytics, classification tasks.

Artificial Intelligence (AI)

- **Definition**:
    - Technology enabling a **machine to simulate human behaviour**.
    - Goal: Create **smart computer systems** to solve complex problems like humans.
    - Broader concept that **encompasses multiple approaches**.
    - Can include systems that **don't learn or adapt over time**.
    - Includes techniques like expert systems and rule-based programming.
    - Can make decisions with limited or no human intervention.
- **AI vs. ML**:
    - **AI**:
        - **Broader concept**.
        - Concerned about **maximising chances of success**.
        - Has a **wide range of scope**.
        - Examples: Siri, customer support chatbots, intelligent humanoid robots.
    - **ML**:
        - **Subset of AI**.
        - Goal: Machines learn from data for **accurate output**.
        - **Limited scope**.
        - Concerned about **accuracy and patterns**.
        - **Always requires large amounts of data** to function.
        - Focuses on algorithms that **improve through data**.
        - Not all AI systems use ML.
- **Traditional AI Systems (non-ML)**:
    - Typically programmed with **explicit rules and logic**.
    - Developers anticipate and code for every situation.
    - Tends to be **more interpretable**.
    - Appropriate if problem has **well-understood rules**.

Machine Learning Models/Techniques

- **Decision Tree**:
    - **Definition**: A **flowchart-like structure for making decisions based on data**.
    - **Components**:
        - **Nodes**: Represent features/attributes.
        - **Branches**: Represent decision rules.
        - **Leaves**: Represent final outcomes/class labels.
    - **How it works**:
        - Collects data with features and outcomes.
        - Algorithm selects feature that best splits data into classes.
        - Data divided, process repeats until conditions met.
        - Used for predictions by traversing from root to leaf node.
    - **Why used by Software Engineers**:
        - **Easy to understand and interpret** (visual structure).
        - **Versatile**: Works with numerical and categorical data.
        - Can capture **complex, non-linear relationships**.
        - Helps **identify most important features** for predictions.
- **Neural Networks (ANN)**:
    - **Definition**: A **computational model inspired by biological brains**.
    - Consists of **layers of interconnected nodes (neurons)** that process information.
    - Useful for tasks like **image recognition, NLP**.
    - Can learn **complex patterns from data** and make predictions/classifications.

    - **Components**:
        - **Node layers**: Input, hidden, and output layers.
        - Each node is a **linear regression model**.
        - **Weights**: Determine influence of each input on output.
        - **Bias/Threshold**: Part of each node.
    - **How it works**:
        - **Feedforward network**: Data passed sequentially between layers.
        - **Training Cycle**:
            - Network learns from data to **minimise prediction error**.
            - Uses a **loss function** to quantify error.
            - Employs **backpropagation** and **gradient descent** to adjust weights and biases.
        - **Execution Cycle**:
            - Uses trained weights to make predictions on **new data**.
            - Input undergoes **forward propagation**.
    - **Types of Neural Networks**:
        - **Convolutional Neural Networks (CNNs)**: Well-suited for **image recognition**.
        - **Recurrent Neural Networks (RNNs)**: Have feedback loops, used with **time series data** for future predictions (e.g., sales forecasting).
    - **Limitations**:
        - Requires **large amounts of data** for effective training.
        - **Resource-intensive training** (needs significant computational power).
        - Often considered **'black boxes'** due to difficulty in interpreting internal workings.
- **Linear Regression**:
    - **Definition**: A mathematical model used to **predict future events**.
    - **Purpose**: Predicts a **continuous numerical value** based on a continuous variable.
    - Finds the **best-fitting line** that minimises overall error across all data points.
    - **Applications**:
        - Predicting house prices.
        - Sales forecasting.
        - Predicting student exam scores.
- **Logistic Regression**:
    - **Purpose**:
        - Generates predictions for **categorical data** (e.g., colour, gender).
        - Ideal for **binary classification problems** (determining probability of belonging to one of two categories).
    - **Data Representation**: Categorical features as **binary values (0 or 1)**.
    - **How it works**: Uses the **sigmoid function** to map input features to a **probability between 0 and 1**.
    - **Applications**:
        - Predicting if a user will continue using an app.
        - Predicting if a password will be cracked (yes/no).
- **K-Nearest Neighbour (KNN)**:
    - **Type**: **Supervised learning algorithm**.
    - Uses **labelled data** to make predictions.
    - **Non-parametric**: Makes no assumptions about data distribution.
    - **How it works**: Finds the **'k' closest points (neighbours)** to a new data point and assigns it to the group most neighbours belong to.
    - Most commonly used for **classification tasks**.
- **Polynomial Regression**:

- **Purpose**: Used when the relationship between variables is **non-linear**.
- Fits a **curve** to the data by **adding powers of the independent variable**.
- **Degree of the polynomial**:
  - The highest power in the equation.
  - Determines the **flexibility of the curve**.
  - Higher degree can fit data closely but may lead to **overfitting**.
  - Example: Degree 1 is a straight line, Degree 2 is a parabola.
- **How it works**:
  - Transforms input features into **polynomial features** (e.g., x, x^2, x^3).
  - Uses these features to fit a regression model.
- **Applications**:
  - Modelling car speed vs. fuel consumption.
  - House prices vs. size (non-linear pricing).
  - Human growth patterns.

DevOps, MLOps, RPA, BPA

- **DevOps**:
  - **Definition**: Methodology combining **software development (Dev) and IT operations (Ops)**.
  - Goal: **Shorten development cycles**, provide **continuous delivery with high quality**.
  - Fosters **collaboration** among roles.
- **MLOps**:
  - Focuses on the **ML life cycle**.
  - **Contrast with DevOps**:
    - **Scope**: MLOps focuses on ML life cycle, DevOps on software development life cycle.
    - **Complexity**: ML models are often more complex, requiring specialised tools.
    - **Data**: ML models rely on data for training/inference, introducing management challenges.
    - **Regulation**: ML models may be subject to regulatory requirements.
  - **MLOps Workflow Phases**: Design, Model Development, Operations.
- **Robotic Process Automation (RPA)**:
  - **Definition**: Software solutions that **replicate human-computer interactions**.
  - Automates **specific, repetitive tasks**.
  - Frees up human resources.
- **Business Process Automation (BPA)**:
  - **Definition**: Strategy for **automating higher-level business workflows**.
  - Often spans multiple departments or systems.
  - Automates entire processes to save time.
- **Other Automation/Analytics Concepts**:
  - **Predictive Analytics**: Uses historical data to **predict future events/behaviours**.
  - **Process Mining**: Systematically **identifies, evaluates, and removes inefficiencies** in business workflows.
  - **Continuous Integration**: Automated process of **merging code changes** from multiple developers into a shared repository regularly.
  - **Cognitive Automation**: Combines **RPA with ML and NLP** to handle unstructured data and make intelligent decisions.

Social and Ethical Issues: Bias in ML/AI Systems

- **Human Bias**:
  - **Definition**: **Subjective judgments and decisions** made by individuals during ML/AI system development.
  - Affects data collection, labelling, model selection, evaluation.
  - **Types**:
    - **Selection Bias**: Developers choose datasets reflecting **own preferences**, leading to models that don't generalise well to diverse populations.
    - **Confirmation Bias**: Developers prioritise results that **confirm existing beliefs**, ignoring contradictory data.
    - **Cognitive Bias**: Subjective judgments lead to biases in **feature selection, model design, evaluation metrics**.
    - **Implicit Bias**: Educators' unconscious biases influencing assessments/grade reporting, affecting training data.
  - **Effects**:
    - **Inaccurate predictions** across diverse scenarios.

        - **Perpetuates/amplifies societal inequalities** (e.g., discriminatory systems).
- **Dataset Source Bias**:
    - **Definition**: Training data is **unrepresentative of the target population** or real-world scenarios.
    - Arises from data collection methods, sampled populations, and data source characteristics.
    - **Types**:
        - **Sampling Bias**: Dataset from a **specific demographic group** may not generalise to others.
        - **Labelling Bias**: Bias introduced during the **labelling process** (annotator biases, inconsistent criteria).
        - **Historical Bias**: Datasets reflecting **historical inequalities** can lead to models reinforcing those biases.
        - **Measurement Bias**: **Inaccuracies or inconsistencies in data** (e.g., grading errors).
    - **Effects**:
        - Leads to decisions that **disadvantage specific groups**.
        - **Erodes trust in technology**.
- **Feedback Loop Bias**:
    - **Description**: Predictions influencing future assessments/opportunities creates a feedback loop that **reinforces existing biases**.
    - **Consequences**: Results in a **cycle of disadvantage** for certain groups.
- **Social and Ethical Issues in Predicting Student Marks (General Concerns)**:
    - **Reliability**: System output must be reliable to allow valid teacher assessment.
    - **Consequences for Action**: May lead teachers to estimate marks rather than encouraging students to catch up.
    - **Potential for Bias**:
        - Considering types of students who frequently miss assessments.
        - Estimates tending higher/lower for certain ability levels.
        - Impact of factors like student's strength in content area.
    - **Data Utilisation**:
        - Previous scores, attendance record, participation in assignments.
        - Average score of classmates.
    - **Conditions for Estimation (Decision Tree Perspective)**:
        - Enough students present to estimate effectively.
        - Not too many gaps in data.
        - Marks consistent enough.
        - Enough assessment tasks completed.
    - **Student Motivation**: Knowledge of model use might lead students to skip difficult tasks for an equal result.
- **Mitigation Strategies for Developers**:
    - Use **Explainable AI (XAI) techniques** to understand model decisions.
    - Provide **training on recognising and mitigating bias** for developers/data scientists.
    - Encourage **diverse teams** in AI development.
    - Conduct **regular audits** to assess bias in datasets and models.
    - Implement **fairness metrics** to evaluate equitable treatment across groups.
    - Mitigate against bias in the **design and development process**.

Secure Software Architecture

Designing Software

- **Benefits of Developing Secure Software**
    - Protects data.
    - Minimises vulnerabilities.
    - Prevents cyber attacks.
    - Safeguards sensitive information.
    - **Data Protection**:
        - Ensures sensitive information (personal, financial, business secrets) is safe from unauthorised access, manipulation, or theft.
        - Achieved via **Encryption** (data at rest and in transit).
        - Achieved via **Access Controls** (role-based access control, RBAC).
    - **Confidentiality**: Sensitive data protected from unauthorised access, maintaining privacy and trust.
        - Ensured by encrypting sensitive data at rest/in transit and secure communication protocols (HTTPS).
    - **Integrity**: Prevents unauthorised modifications or tampering; data remains accurate and reliable.
        - Ensured by data validation, integrity checks, checksums, digital signatures, and hash functions.
    - **Availability**: Data and services accessible to legitimate users; mitigates downtime.
        - Ensured by robust backup/recovery mechanisms, redundancy, load balancing, and disaster recovery.
    - **Compliance with Regulations**: Helps meet data protection regulations (GDPR, HIPAA, PCI DSS); avoids legal consequences.
    - **Customer Trust**: Builds confidence, enhances customer loyalty and satisfaction.
    - **Business Continuity**: Reduces data loss/service disruptions; ensures continuity of operations.
    - **Minimising Cyber Attacks and Vulnerabilities**: Secure design features protect data, minimise vulnerabilities, prevent attacks.
        - Includes secure coding practices and regular system updates.
    - **Prevent Data Breaches**: Implements security measures to protect sensitive information.
    - **Mitigate Cyber Threats**: Reduces likelihood of malware, ransomware, phishing, Denial of Service (DoS) attacks.
    - **Cost Savings**: Addressing security vulnerabilities early is more cost-effective than post-deployment fixes.
    - **Competitive Advantage**: Differentiates organisations by demonstrating commitment to cybersecurity.
    - **Vulnerabilities**: Weaknesses in software that attackers can exploit.
        - Often from poor coding, outdated libraries, or misconfigurations.

        - Avoided by input validation, parameterised queries, and correct output encoding.
- **Fundamental Software Development Steps to Develop Secure Code**
    - The Software Development Life Cycle (SDLC) embeds security from the beginning.
    - **Requirements Definition**: Gathers and analyses what software must achieve, user needs, and functionality.
        - Emphasises non-functional security requirements (e.g., password hashing).
        - Techniques: interviews, surveys, use case scenarios, threat modeling.
    - **Determining Specifications**: Translates requirements into clear, actionable details on system function and security.
        - Considers data handling, access controls, and attack protection.
        - Example specifications: strong password hashing (SHA-256, bcrypt), rate limiting, HTTPS transmission, PCI DSS compliant services, tokenisation.
    - **Design**: Defines system architecture and key components (database models, views, web page structures).
        - **Key Design Principles for Secure Architecture**:
            - **Modularity**: Breaking software into smaller, manageable pieces.
            - **Abstraction**: Hiding complex details, exposing essential parts.
            - **Encapsulation**: Keeping related data and functions together.
            - **Reusability**: Designing components for reuse.
            - **Maintainability**: Ensuring software is easy to maintain and update.
        - **Model-View-Controller (MVC) Pattern**: Separates concerns into Model (data/business logic), View (user interface), and Controller (intermediary).
            - Django implements as Model-View-Template (MVT).
    - **Development**: Actual code is written according to the design plan.
        - Includes managing user security preferences.
    - **Integration**: All modules are integrated and tested to ensure they work together.
        - Security considerations: assessing existing system security, aligning new requirements, modular approach, secure database integration, thorough testing and validation.
    - **Testing and Debugging**: Essential for identifying and fixing vulnerabilities, errors, and weaknesses before deployment.
        - Includes functional and security tests (e.g., preventing unauthorised access via SQL injection).
    - **Installation**: Process of setting up software securely on a server or device.
        - **Key Practices**:
            - Use of **Secure Connections (HTTPS)** with SSL/TLS certificates.
            - **Environment Isolation** for development, testing, and production.

        - **Database Configuration** with strong credentials and restricted external access.
        - **Access Control and Permissions** following the principle of least privilege.
    - **Maintenance**: Regular updates, bug fixes, new features, and essential security patches in response to discovered vulnerabilities.
        - Ensures the system remains secure post-deployment via regular updates, immediate patching, and continuous monitoring.
- **How Capabilities and Experience of End Users Influence Secure Design Features**

    - Understanding user capabilities, expectations, and behaviours is crucial for designing secure systems.
    - Misalignment can lead to misuse and increased vulnerabilities.
    - User-Centered Design (UCD) ensures software is easy to use, meets needs, and minimises security risks.
    - **Key Principles of UCD for Secure Software**:
        - **Simplicity**: Simple interfaces reduce user errors leading to security issues.
        - **Clarity**: Clear instructions and feedback help users make correct choices.
        - **Transparency**: Users understand why security actions are needed (e.g., two-factor authentication).
    - **Common User Behaviours Affecting Security and Solutions**:
        - **Weak password practices**: Implement strong password policies.
        - **Phishing and social engineering**: Educate users on identifying suspicious messages.
        - **Failure to update software**: Encourage automatic updates and explain their importance.
        - **Bypassing security protocols**: Design user-friendly protocols, offering flexible options.
    - **How User Capabilities/Experience Influence Secure Design Features**:
        - **User-Friendly Authentication**: Designing mechanisms that are secure yet easy to use, offering Multi-Factor Authentication (MFA) or simpler methods.
        - **Role-Based Access Control**: Understanding user roles helps implement granular access controls.
        - **Training and Awareness**: Providing educational resources on secure practices, especially for less experienced users.
        - **User Feedback and Testing**: Involving users in testing provides insights for refining security features.
        - **Simplified Security Controls**: For less experienced users, prevents security features from becoming barriers to usability.

Developing Secure Code

- **Fundamental Software Design Security Concepts**

    - The **CIA triad** (Confidentiality, Integrity, Availability) and **AAA** (Authentication, Authorisation, Accountability) form the foundation.
    - **Confidentiality**: Sensitive data is kept private and accessible only to authorised users.
        - Ensures sensitive information is encrypted and securely stored.
        - **Importance**: Protects user credentials and personal data, maintains privacy and trust.
        - **How to Ensure**: Encrypt sensitive data at rest/in transit, implement secure communication protocols (HTTPS).
    - **Integrity**: Data remains accurate, consistent, and trustworthy, preventing unauthorised modification.
        - Verifies data integrity to prevent tampering or alteration.
        - **Importance**: Ensures data remains accurate, consistent, and trustworthy.
        - **How to Ensure**: Implement checksums, digital signatures, and hash functions; use encryption for data integrity during transmission.
    - **Availability**: Information and resources are available to authorised users when needed.
        - Ensures system services and resources are accessible.
        - **Importance**: Prevents downtime, ensures continuity of services, maintains user satisfaction.
        - **How to Ensure**: Design with redundancy, load balancing, disaster recovery; monitor performance proactively.
    - **Authentication**: Verifies the identity of a user before granting access.
        - **Importance**: Prevents unauthorised access and ensures users are who they claim to be.
        - **How to Ensure**: Implement strong authentication mechanisms (passwords, 2FA, biometrics) and secure protocols.
    - **Authorisation**: Ensures only authorised users can access certain parts of the application.
        - Enforces access control mechanisms to determine user permissions.
        - **Importance**: Prevents unauthorised activities and protects sensitive data.
        - **How to Ensure**: Implement Role-Based Access Control (RBAC) or Attribute-Based Access Control (ABAC); regularly review permissions.
    - **Accountability**: Every action within a system can be traced back to a responsible individual.
        - Implements logging and auditing mechanisms to track user activities and security events.
        - **Importance**: Establishes traceability and responsibility in case of security incidents.
        - **How to Ensure**: Maintain logs of user actions, access attempts, and security events; regularly review audit logs.
- **Security Features Incorporated into Software**

    - **Data Protection**: Secure password hashing (Bcrypt), encrypting sensitive data at rest and in transit (TLS).
        - Includes data minimisation and retention policies.

    - **Security Measures**: Input validation and sanitisation to prevent vulnerabilities like SQL injection and XSS.
        - Uses parameterised queries, prepared statements, secure coding, and robust error handling.
    - **Privacy Protection**: Clear privacy policy, obtaining user consent, transparency on data handling.
        - Uses privacy-enhancing technologies like anonymisation and pseudonymisation.
    - **Regulatory Compliance**: Aligning security measures with regulations (e.g., Privacy Act 1988 Australia).
        - Ensuring user data rights (access, rectification, deletion) and regular security audits.
    - **User Authentication and Authorisation**: Strong mechanisms like MFA (e.g., Google Authenticator).
        - Enforcing access controls, monitoring, and logging user access.
- **Contribution of Cryptography and Sandboxing to 'Security by Design' (SbD)**

    - **Security by Design (SbD)**: Integrates security controls into software from the design phase, making security a core element.
    - **Cryptography**: Process of encoding information so only the intended recipient can read it.
        - Ensures data **confidentiality, integrity, and authenticity**.
        - Uses strong encryption algorithms to protect sensitive data at rest and in transit.
        - Hashing converts plain passwords into a non-reversible form before storage.
    - **Sandboxing**: A security method using an isolated environment to test and analyse code safely without affecting the main system.
        - Limits potential impact of security breaches by containing risky operations.
        - Examples: web browser sandboxing, sanitising user input, isolating third-party API calls, running untrusted scripts in controlled environments.
        - Development and testing environments can also be sandboxed.
- **'Privacy by Design' (PbD) Approach**

    - **Privacy by Design (PbD)**: A proactive approach integrating privacy into the design and development of systems from the outset.
    - **Proactive Not Reactive Approach**: Anticipates and addresses potential security and privacy issues during design, preventing data compromise.
    - **Embed Privacy into Design**: Privacy protections built-in by default.
        - Includes data minimisation, anonymising sensitive information, and secure communication.
    - **Respect for User Privacy**: Individuals' interests are paramount in design and implementation.
        - Achieved by strong privacy defaults, user-friendly options, appropriate notice, and user control over data (e.g., account deletion).

Testing and Evaluating Software Security and Resilience

- **Determining Vulnerabilities**: Identifying weaknesses (e.g., SQL injection, XSS, weak authentication).
    - Through automated tools, manual inspection, and simulated attacks.
- **Hardening Systems**: Adopting best practices for securing software systems.
- **Handling Breaches**: Involves detection, containment, eradication, recovery, business continuity, and disaster recovery.
    - An incident response plan aids quick detection, containment, and mitigation.
- **Maintaining Business Continuity**: Organisation's ability to maintain critical operations during and after a disruptive event.
    - Secure software ensures resistance to attacks, minimising incident impact.
    - Includes data integrity/availability and proactive threat mitigation.
- **Conducting Disaster Recovery**: Focuses on restoring systems and operations after a major failure or attack.
    - Secure software architecture is fundamental for fast, efficient, and complete recovery.
    - Involves reliable backup systems and rapid response.
    - A broader strategy that restores IT infrastructure, systems, and operations.
    - Involves predefined Recovery Time Objective (RTO) and Recovery Point Objective (RPO).
- **Strategies Used by Software Developers to Manage Security**
    - **Code Review**: Systematic examination of code to identify bugs, improve quality, and ensure secure coding standards.
        - Peer reviews identify insecure code, logic flaws, weak error handling, or hard-coded sensitive data.
    - **Static Application Security Testing (SAST)**: Tools analyse source code without executing to identify vulnerabilities.
        - **Advantages**: Detects vulnerabilities early, provides immediate feedback, best for finding insecure coding practices.
        - Proactive approach, reducing breach likelihood and future fixing costs.
    - **Dynamic Application Security Testing (DAST)**: Tools test the running application, simulating attacks from an external perspective.
        - **Advantages**: Identifies vulnerabilities only visible at runtime (e.g., SQL injection, XSS, insecure session management), tests the system as a whole, offers insights into attacker exploitation.
    - **Vulnerability Assessment**: Systematic process of scanning a web application to identify weaknesses or flaws that could be exploited.
        - Identifies, quantifies, and prioritises security vulnerabilities to assess risks and recommend mitigation.
    - **Penetration Testing (Pen Testing)**: Proactive approach simulating real-world attacks to evaluate security.

        - Actively exploits vulnerabilities to understand their potential impact.
        - Uncovers flaws not obvious during routine testing and provides insights into how attackers could breach a system.
- **Defensive Data Input Handling Practices**

    - Defensive programming ensures systems are robust and secure against unexpected or malicious input.
    - **Input Validation**: Verifying that user input meets the system's expected format and constraints before processing.
        - Prevents malformed data and blocks common attacks (e.g., SQL injection, buffer overflows).
        - **Types**: Client-side (for feedback, not security-reliant) and Server-side (critical for security).
        - **Best Practice**: Validate on both client and server, use built-in validation libraries, never trust user input, use whitelist validation.
    - **Sanitisation**: Cleaning or escaping user input to remove harmful characters or scripts.
        - Critical for preventing XSS and SQL injection attacks.
        - For XSS, special characters are escaped.
        - **Importance**: Cleans data for processing/storage, preventing malicious code execution.
    - **Error Handling**: Detecting, managing, and responding to errors during program execution.
        - Improper handling can lead to security vulnerabilities or data exposure.
        - **Key Principles**: Do not expose sensitive information (use generic messages), differentiate between user and developer messages (log detailed errors internally), graceful error recovery, log errors securely.
        - **Best Practices**: Use try-except blocks, avoid overly detailed error messages, fail safely, log errors securely.
- **Safe Application Programming Interface (API) Design and Implementation**

    - APIs facilitate communication but can introduce vulnerabilities if not properly secured.
    - **Common API Vulnerabilities**:
        - Lack of Authentication and Authorisation.
        - Data Exposure (returning too much sensitive information).
        - Insecure Communication (lack of encryption).
        - Injection Attacks (malicious code injected into requests).
        - Lack of Rate Limiting (abuse through brute-force/DoS attacks).
    - **Best Practices for Securing APIs**:
        - Always require **Authentication and Authorisation** (e.g., OAuth 2.0).
        - Use **HTTPS** to encrypt all API traffic.
        - Implement **Input Validation and Sanitisation**.
        - **Limit Data Exposure**, only returning necessary data.
        - Implement **Rate Limiting** to prevent brute-force/DoS attacks.
        - Use **API Keys and Tokens** with expiration.
        - **Audit and Monitor API Usage**, logging suspicious activity.

- **Code Considering Efficient Execution for the User**
    - **Memory Management**: Ensures computer resources are used wisely.
        - Involves avoiding memory leaks, making operations faster, handling more users, maintaining program stability, and protecting against attacks.
        - Ensures sensitive data in memory (passphrases, tokens) is not exposed and is cleared when no longer needed.
    - **Session Management**: Tracks user interactions with a website or application.
        - Enables personalised experiences, authenticates users, remembers actions, prevents data loss, manages interactions, enhances performance.
        - **Key Concepts**: Session IDs (unique, random, securely stored), Session Timeout (automatic expiration), Session Regeneration (new ID on key events), Secure Cookies (HttpOnly and Secure flags).
        - **Key Vulnerabilities**: Session Hijacking (attacker steals ID), Session Fixation (attacker forces known ID).
    - **Exception Management**: Allows a program to handle unexpected problems gracefully, preventing crashes.
        - Provides guidance to users when errors occur, maintains program stability, aids in debugging, and helps protect from security threats.
- **Secure Code to Minimise Vulnerabilities in User Action Controls**
    - **Broken Authentication and Session Management**: Vulnerabilities allowing unauthorised access.
        - Attackers can steal credentials, exploit session weaknesses, or bypass mechanisms.
        - **Prevention**: Enforce strong password policies, use Multi-Factor Authentication (MFA), and implement secure session management (HttpOnly/Secure cookies, session ID regeneration).
    - **Cross-Site Scripting (XSS)**: Attackers inject malicious scripts into web pages viewed by other users.
        - Scripts execute in the user's browser, potentially stealing sensitive information or performing actions on the user's behalf.
        - **Types**: Stored XSS, Reflected XSS, and DOM-based XSS.
        - **Prevention**: Sanitise input (escape/remove special characters) before storage/display; use output encoding to treat data as text.
    - **Cross-Site Request Forgery (CSRF)**: An attack where an attacker tricks a user into performing unintended actions while authenticated.
        - The target website processes the request as legitimate due to session cookies.
        - **Prevention**: CSRF Tokens (unique, secret value included in forms, verified on submission) and SameSite Cookies (prevents browsers from sending cookies with cross-site requests).
    - **Invalid Forwarding and Redirecting**: Information not found in sources.
    - **Race Conditions**: Occur when multiple operations access and modify the same shared resource concurrently, and the final outcome depends on the unpredictable order of execution.

        - Can lead to unexpected and potentially harmful results, including security vulnerabilities.
        - **Prevention**: Use locks (e.g., `threading.Lock` in Python) to ensure only one thread accesses a critical section of code at a time.
- **Secure Code to Protect User File and Hardware Vulnerabilities from File Attacks and Side Channel Attacks**
    - **File Attacks (File System Security)**: Improper handling of file uploads, inadequate permissions, or unsafe file access can lead to vulnerabilities.
        - **Common File System Vulnerabilities**: Malicious File Uploads, Path Traversal (Directory Traversal), Improper File Permissions, File Name Injection.
        - **Prevention**: File Type Validation, File Name Sanitisation, Limit File Permissions, Random File Names, Store Files in Secure Directories, Implement Access Control, Scan Uploaded Files for Malware, Limit File Size, Encryption (AES with PBKDF2).
    - **Side Channel Attacks**: Gather information from the physical implementation of a computer system, rather than exploiting software vulnerabilities directly.
        - Exploit indirect information leaked during normal operations (e.g., timing, power consumption) to deduce sensitive data (e.g., encryption keys).
        - **Types**: Timing Attacks, Power Analysis Attacks, Electromagnetic and Acoustic Attacks, Cache Attacks, Firmware Attacks, Cold Boot Attacks, USB-Based Attacks.
        - **Mitigation**: Constant-Time Algorithms, Blinding Techniques, Random Delays, Shielding and Filtering, Proper Cache Management, Memory Safety, Hardware Isolation (Trusted Execution Environments), Access Controls, Firmware Updates, Secure Boot, Physical Security, Hardware-Based Encryption (TPM), Monitor Power and Temperature.

Impact of Safe and Secure Software Development

- **Benefits of Collaboration to Develop Safe and Secure Software**
    - Collaboration is a critical component of secure software development.
    - Teams pool expertise, catch mistakes early, and identify insecure code.
    - Version control systems (e.g., GitHub) enable effective collaboration.
    - **Considering Various Points of View**: Each team member brings unique skills and experiences, identifying potential security issues.
    - **Delegating Tasks Based on Expertise**: Leveraging unique skills leads to higher quality and more secure end-products.
    - **Quality of the Solution**: Continuous feedback and review allow for timely identification and resolution of security issues.
- **Benefits to an Enterprise of Implementing Safe and Secure Development Practices**
    - **Improved Products or Services**: Enhances quality and reliability, leading to increased customer trust and satisfaction.

    - **Influence on Future Software Development**: Sets a precedent within the organisation for future projects, promoting a proactive security approach.
    - **Improved Work Practices**: Involves structured processes and guidelines, leading to more efficient work practices.
        - Methodologies like Agile and DevOps (DevSecOps) integrate security into every stage.
    - **Productivity**: Addresses security concerns early, avoiding costly rework and time-consuming fixes later.
        - Includes automating repetitive tasks to reduce human error.
    - **Business Interactivity**: Better positioned to engage with partners, clients, and stakeholders securely.
        - Opens new opportunities for collaboration and business growth.
        - Secure software has been pivotal in making remote work viable.
- **Social, Ethical, and Legal Issues and Ramifications**

    - **Employment**: Creates jobs for skilled cybersecurity professionals (e.g., cybersecurity engineers, secure software developers, compliance officers).
        - **Social Impact**: Automation and AI can lead to job displacement and evolving skill requirements.
    - **Data Security**: Prioritises safeguarding personal data from unauthorised access.
        - Poor security can lead to identity theft, financial loss, or emotional distress.
        - Ethical developers must adhere to privacy laws and ensure user control over data.
    - **Privacy**: Safeguarding personal information and data from unauthorised access and misuse.
        - Developers are ethically bound to design software adhering to privacy laws (e.g., GDPR).
        - **Ethical Issue**: Ensuring data is not shared without consent.
        - **Legal Issue**: Compliance with data protection regulations.
    - **Copyright**: Legal framework granting creators exclusive rights to their original works, including software code.
        - Prevents others from copying, distributing, or using without permission.
        - **Legal Issue**: Protecting original works and preventing unauthorised copying or distribution.
    - **Intellectual Property (IP)**: Creations of the mind (inventions, designs, symbols), extending to code, algorithms, and software products.
        - **Patents** protect unique software methods; **Trademarks** protect brand identity.
        - **Licensing** dictates how software can be used, modified, or distributed (open-source vs. proprietary).
        - **Legal Issue**: Safeguarding intangible assets like software code, patents, and trade secrets.
    - **Digital Disruption**: Disruption of traditional business models and industries by digital technologies.

- Insecure software can exacerbate the digital divide by making services inaccessible.
- **Social Impact**: Rise of e-commerce platforms disrupting traditional retail.

Programming for the Web

Web Programming Applications

- **Interactive Websites/Webpages**:
    - Enable users to **browse, communicate, and interact** with information online.
    - Involve **dynamic functionality**, not just static content.
    - Focus on **user engagement, data management, and productivity**.
- **E-commerce Platforms**:
    - Facilitate **buying and selling goods or services** over the internet.
    - Enabled by **widespread internet access, mobile technology, secure payment gateways, and UX innovations**.
- **Progressive Web Apps (PWAs)**:
    - Combine features of **web and mobile applications**.
    - **Installable** on devices without app stores.
    - Offer **faster, more reliable experience** even in poor network conditions.
    - Provide **offline functionality**.
    - Built using **HTML, CSS, JavaScript, and Web APIs**.
    - **Adapt to different screen sizes** and always stay up-to-date.
    - Can offer **push notifications** and **full-screen mode**.
    - Recognisable by: installation prompt, home screen icon, full-screen mode, push notifications, offline functionality, and **responsive design**.
    - Built with **progressive enhancements** (basic version works on all browsers, enhanced for modern ones).

Data Transmission on the Internet

- **Data Packets**:
    - When data is sent, it's broken into smaller units called **data packets**.
    - Makes transmission **efficient, reliable, and scalable**.
    - Each packet contains part of the data plus information for destination and reassembly.
    - **Components**:
        - **Header**: Contains metadata like **Source IP, Destination IP, Sequence Number, Protocol Information, Error Checking Information**.
        - **Payload**: The **actual data** being sent.
        - **Trailer**: Indicates end of packet, potentially additional error-checking data.
    - Travel using **packet switching**.
        - **Packet Switching**: Fundamental technology for efficient, reliable internet communication.
        - Data divided into **small, manageable units** (typically 1,500 bytes) and transmitted independently.
        - Routers examine header to direct packets, allowing **dynamic routing**.

        - **Key features**: Independent packet routing, packet reassembly via sequence numbers, error detection and retransmission, shared network resources.
        - **Advantages**: **Efficiency, reliability, error recovery, scalability**.
        - **Disadvantages**: Variable delays, potential for packet loss (though rare).
        - **Applications**: The Internet itself, Cellular Networks, Email Systems.
- **Internet Protocol (IP) Addresses**:
    - **IP**: Principal communication protocol for **relaying data packets** across network boundaries. Establishes the Internet's routing function.
    - **IP Address**: Acts like a postal address for sending/receiving data; each device has a unique IP address.
    - **IPv4**:
        - Fourth and first widely deployed version of IP.
        - Uses a **32-bit address scheme**, providing over 4.29 billion unique addresses.
        - Written as **four decimal numbers separated by periods** (e.g., 192.168.1.1), each 0-255.
        - Running out of addresses due to device growth.
    - **IPv6**:
        - Developed to address IPv4 limitations.
        - Uses a **128-bit address**, offering practically unlimited addresses.
        - Represented as **eight groups of four hexadecimal digits separated by colons**.
        - **Key differences (IPv4 vs. IPv6)**: Writing format (decimal vs. hexadecimal), number of addresses (limited vs. practically unlimited), IPv6 has more advanced features like built-in security and better mobile support.
- **Domain Name System (DNS)**:
    - **DNS**: System that **translates user-friendly domain names** (e.g., "www.example.com") **into IP addresses**. Acts as the internet's "address book".
    - **How DNS Works**:
        - User enters domain; browser requests IP address.
        - Browser contacts a DNS resolver (often by ISP).
        - Resolver checks cache; if not found, queries a **hierarchy of DNS servers** (root, TLD, authoritative).
        - Authoritative server returns the IP address.
        - Resolver sends IP to browser, enabling connection to website's server.
    - **Common DNS Records**:
        - **A record**: Maps domain to an **IPv4 address**.
        - **AAAA record**: Maps domain to an **IPv6 address**.
        - **CNAME record**: Maps a domain to **another domain**.
        - **MX record**: Specifies **mail servers** for receiving email.
    - **Port**: DNS typically uses **Port 53**.

Web Protocols and Their Ports

- **Web Protocols**: Essential for **facilitating communication** over the internet, each serving a specific purpose and operating on designated ports.
- **HTTP (Hypertext Transfer Protocol)**:
    - Allows web browsers to **fetch resources** (like HTML pages) from web servers.
    - Defines request/response format and transmission.
    - **Stateless Protocol**: Each client request is independent; server doesn't remember previous requests.
    - **Unsecured**: Data can be intercepted.
    - **Port**: Typically uses **Port 80**.
- **HTTPS (Hypertext Transfer Protocol Secure)**:
    - **Secure version of HTTP**.
    - Uses **SSL/TLS encryption** to ensure data security, especially for sensitive information.
    - **Security Indicator**: Padlock icon next to URL in browser.
    - **Crucial for secure web applications**: Protects against eavesdropping and man-in-the-middle attacks.
    - **Port**: Commonly uses **Port 443**.
- **TCP/IP (Transmission Control Protocol/Internet Protocol)**:
    - **Foundational communication protocol suite** for the internet.
    - **TCP**: Responsible for **reliable data transmission**. Breaks data into packets, sends, and reassembles. **Connection-oriented protocol**.
    - **IP**: Handles **addressing and routing** data packets.
    - **Difference**: IP defines the address, TCP transports and routes data, ensuring reliable delivery.
    - **Transport Protocol**: Web servers primarily use **TCP** for reliable, ordered, error-checked delivery.
- **FTP (File Transfer Protocol) and SFTP (Secure File Transfer Protocol)**:
    - **FTP**: Used for **transferring files** between client and server.
    - **FTP Ports**: Uses **Ports 20 (data transfer) and 21 (control)**.
    - **SFTP**: **Secure version of FTP**, offering **encrypted file transfers over SSH**.
    - **SFTP Port**: Uses **Port 22** (default for SSH).
    - **Preferred over FTP for sensitive data** due to confidentiality and data integrity.
- **SSL (Secure Sockets Layer) and TLS (Transport Layer Security)**:
    - **SSL**: Original encryption protocol, largely replaced by its successor **TLS**.
    - **Encryption**: Uses cryptographic algorithms to **scramble data**, preventing interception.
    - **Authentication**: Authenticates the server's identity, preventing man-in-the-middle attacks.
    - **Ports**: Do not have specific ports, but are used with protocols like HTTPS (443), SMTP (25, 587), and IMAP (993) to encrypt communication.
- **SMTP (Simple Mail Transfer Protocol), POP3 (Post Office Protocol version 3), IMAP (Internet Message Access Protocol)**:
    - **SMTP**: Used for **sending email messages** between servers.
    - **SMTP Port**: Default is **Port 25**.
    - **POP3 and IMAP**: Email retrieval protocols used by email clients.

    - **POP3**: **Downloads emails to user's device**, often removing from server. Uses **Port 110**.
    - **IMAP**: Allows **viewing emails remotely on the server**, synchronising across devices. Uses **Port 143**.
    - **Secure Versions**: POP3S uses **Port 995**, IMAPS uses **Port 993**.

Securing the Web

- **Web Security**: Involves implementing technologies and practices to ensure **confidentiality, integrity, and availability of data and systems**.
- **Secure Sockets Layer (SSL) Certificates**:
    - Digital files that **digitally bind a cryptographic key to an organisation's details**.
    - Enable **secure connections (HTTPS)** by encrypting data between web server and browser.
    - Authenticates a website's identity and enables an encrypted connection.
    - Issued by a **Certificate Authority (CA)**, a trusted third-party.
    - **How SSL/TLS Certificates Work**:
        - Browser requests secure connection.
        - Server sends its SSL certificate (including public key and digital signature).
        - Browser verifies certificate's validity.
        - **SSL/TLS handshake** generates a **session key** (temporary, symmetric key).
        - Data is then securely communicated using the session key.
    - Ensures **secure browsing (encryption), authentication (identity verification), and data integrity**.
- **Encryption Algorithms**:
    - Convert readable data (**plaintext**) into an unreadable format (**ciphertext**).
    - **Symmetric Encryption**: Uses the **same key** for encryption and decryption (e.g., AES).
    - **Asymmetric Encryption**: Uses **two keys** – a **public key** for encryption and a **private key** for decryption (e.g., RSA). Used in SSL/TLS for secure key exchange.
- **Encryption Keys**:
    - Sequences of characters determining encryption/decryption output. Must be kept secure.
    - **Public Key**: Shared openly to encrypt a message.
    - **Private Key**: Kept secret, used for decrypting messages encrypted with the public key.
    - Longer keys (e.g., 256-bit) provide stronger security.
    - **Key Exchange**: Process of securely sharing keys over an unsecured network. SSL/TLS uses asymmetric encryption for key exchange and symmetric for data transfer.
- **Plaintext and Ciphertext**:
    - **Plaintext**: Original, readable message or data.
    - **Ciphertext**: Unreadable format of data after encryption.
- **Authentication and Authorisation**:

    - **Authentication**: **Verifies who a user is** (e.g., username, password, biometrics).
    - **Authorisation**: **Determines what an authenticated user can access** (permissions).
    - Together, prevent **unauthorised access**.
    - **OAuth**: Authorisation framework for applications to access resources on behalf of users without sharing passwords.
    - **OpenID Connect (OIDC)**: Builds on OAuth by adding authentication capabilities, used for single sign-on (SSO).
- **Hash Values**:
    - A hash function produces a **fixed-size string of characters (a 'hash')** from input data. Unique to input data.
    - **Key properties**: **Deterministic**, **Irreversible**, and **Collision-Resistant**.
    - Used for **verifying data integrity** (e.g., file checksums) and **securely storing passwords** (storing hashed passwords instead of plaintext).
- **Digital Signatures**:
    - Verify the **authenticity and integrity** of a message or document.
    - Use a sender's **private key** to create a signature, verified by recipient's **public key**.
    - Provide: **Authentication, Integrity, and Non-Repudiation**.

Effect of Big Data on Web Architecture

- **Big Data**: Refers to **massive datasets** that cannot be managed effectively using traditional data processing techniques.
- Encompasses **Volume, Variety, Velocity, and Veracity (the four Vs)**.
- Has significantly impacted web architecture, leading to more **scalable, flexible, and real-time data processing systems**.
- **The Four Vs of Big Data**:
    - **Volume**: The **amount of data** generated (e.g., social media posts, videos).
    - **Variety**: The **types and formats of data** collected (structured, semi-structured, unstructured).
    - **Velocity**: The **speed at which data is generated and processed** (e.g., real-time user clicks, IoT sensor data).
    - **Veracity**: The **quality, trustworthiness, and reliability of the data**.
- **Opportunities from Big Data**:
    - **Customer Personalisation**: Analysing data for personalised recommendations.
    - **Predictive Analytics**: Anticipating needs, predicting failures, or forecasting demand.
    - **Enhanced Decision Making**: Combining data for more informed decisions.
- **Challenges**: Storage and processing, data integration and analysis, real-time processing.
- **Effect on Web Architecture**: Driven evolution towards more **scalable, flexible, and real-time data processing systems**, involving data mining, metadata, and streaming service management.
- **Data Mining**:

    - Process of **discovering patterns, trends, and insights from large datasets**.
    - Helps to predict future trends and make informed business decisions.
    - **Applications**: Describing target datasets, predicting outcomes, detecting fraud, learning about user bases.
    - **Key Data Mining Techniques**:
        - **Association**: Recommends items based on what users consume together.
        - **Classification**: Categorises users based on their habits.
        - **Clustering**: Groups users based on shared habits without predefined categories.
        - **Predictive Analytics**: Anticipates user behavior to recommend new content or predict trends.
    - **Benefits in Streaming**: Enhanced personalisation, optimised content delivery, and business intelligence.
    - **Challenges in Streaming**: Data privacy concerns, scalability, and real-time analysis.
- **Metadata**:
    - "**Data about data**". Provides critical information about a dataset's characteristics.
    - Easier to **organise, search, and understand** large volumes of information.
    - **Purposes**: Descriptive, Structural, and Administrative metadata.
    - **Relationship with Big Data**: Critical for managing overwhelming information in web applications.
    - **Effect on Web Applications**: Enables performance and scalability, data processing and personalisation, facilitates search and filtering, and raises security and privacy concerns.
    - **Role in Web Architecture**: Data organisation and management, enhanced searchability, and data governance/compliance.
- **Streaming Service Management**:
    - **Real-time data streaming**: Processing and analysing data as it is generated, without first storing it. Crucial for applications requiring immediate responses. Associated with high-velocity data.
    - Manage enormous amounts of data from millions of users daily.
    - **How Streaming Services Manage Big Data**:
        - **Data Storage and Retrieval**: Store massive volumes of media and user interactions; use distributed server networks (e.g., Content Delivery Networks or CDNs) to minimise latency.
        - **Real-Time Data Processing**: Process data instantly for seamless experiences (e.g., video/audio without interruptions). Rely on caching, buffering, and CDNs.
        - **Data Compression and Encoding**: Reduce file sizes without sacrificing quality for efficient bandwidth use. **Adaptive bitrate streaming** adjusts stream quality in real-time based on internet speed.
        - **Personalisation and Recommendations**: Analyse user data with machine learning algorithms and data mining. Techniques include **collaborative filtering** and **content-based filtering**.

Designing Web Applications

- **World Wide Web Consortium (W3C)**:
    - International organisation that develops **open standards** to ensure the long-term growth of the web.
    - Defines protocols and guidelines for **seamless web operation across devices and platforms**.
    - Its standards (W3C Recommendations) cover HTML, CSS, XML, SVG, and the Semantic Web, ensuring **accessibility, interoperability, and security**.
- **W3C's Web Accessibility Initiative (WAI)**:
    - Committed to making the web **accessible to everyone**, including individuals with disabilities.
    - **Web Content Accessibility Guidelines (WCAG)** provide a framework for ensuring websites are usable by people with various disabilities.
- **W3C's Internationalisation (I18n)**:
    - Standards help developers create PWAs that support **multilingual content** and accommodate diverse cultural conventions.
- **W3C's Web Security**:
    - Standards (e.g., HTTPS, CSP, CORS, secure authentication) help developers build **secure PWAs** that mitigate common security threats.
- **W3C's Privacy**:
    - Provides privacy standards and guidelines for designing PWAs that **respect user privacy rights** and comply with regulations.
- **W3C's Machine-Readable Data**:
    - Standards for representing and exchanging **machine-readable data**, enhancing interoperability and automation. Makes PWA content easier to understand and process by search engines.
    - **XML** (Extensible Markup Language) and **SVG** (Scalable Vector Graphics) are W3C standards.

Web Development System Elements

- A web application's structure consists of **front-end (client-side) and back-end (server-side)** components.
- Basic web app interaction involves a **request and response**.
    - **Client Request**: User initiates request; browser sends **HTTP request** (URL, HTTP headers). Requests can be **GET** (retrieve data) or **POST** (send data).
    - **Server Processing**: Web server receives request; retrieves static resources quickly; for dynamic pages, runs **server-side code** (e.g., Python, PHP, Node.js), often interacting with a **database**.
    - **Server Response**: Server sends **HTTP response** (status code, response data like HTML/JSON, headers).
    - **Browser Renders**: Browser parses HTML to create **Document Object Model (DOM)**, loads resources (CSS, JavaScript, images), executes client-side scripts for interactivity.
- **Static vs. Dynamic Elements**:
    - **Code**:
        - **Static Code**: Content hard-coded into HTML, unchanging.

        - **Dynamic Code**: Content generated from a database or CMS by server-side languages, changing based on conditions.
    - **Delivery**:
        - **Static Delivery**: Page pre-generated and stored on a server/CDN, sent directly.
        - **Dynamic Delivery**: Server generates page on-the-fly in response to requests.
    - **Client Browser Interaction**:
        - **Static Client Browser**: Content remains the same, no interaction beyond viewing.
        - **Dynamic Client Browser**: Uses **JavaScript** to make real-time changes based on user interactions or asynchronous data fetching (AJAX).
- **Client-side (Front-End) Web Programming**:
    - Code that runs on the **user's device**, typically within a web browser.
    - Creates the **interactive parts** of websites.
    - **Languages/Technologies**: **HTML** (structures), **CSS** (styles), **JavaScript** (primary language for interactivity, DOM manipulation, AJAX, input validation).
    - **Less secure** (visible to users). **Fast for small tasks** (no server communication).
- **Server-side (Back-End) Web Programming**:
    - Handles **operations that occur on a web server**.
    - Processes requests, manages databases, security, file operations.
    - **Languages/Technologies**: **Python** (Django, Flask), **Node.js**, **PHP**, **Ruby** (Ruby on Rails), **Java** (Spring Boot).
    - **More secure** (hidden from users). **Generally slower** (due to server-client communication).
- **Interfacing with Databases (SQL or NoSQL)**:
    - **Database**: Structured collection of data.
    - **Relational Databases (SQL)**:
        - Store data in **tabular format**.
        - Organised into **tables with rows and columns**, with relationships through keys (primary, foreign).
        - Ideal for **structured data**. Use **SQL for queries**.
        - Follow **ACID principles** (Atomicity, Consistency, Isolation, Durability) for reliability/integrity.
        - **Require a schema to be defined**.
        - **Examples**: MySQL, PostgreSQL, SQLite, Microsoft SQL Server, Oracle.
        - **Entities**: Real-world objects/concepts. **Attributes**: Properties of the entity. **Instances**: Actual data records (rows).
        - **Entity Relationships**: One-to-One, One-to-Many (ForeignKey), Many-to-Many (ManyToManyField).
    - **NoSQL Databases**:
        - **Non-relational**, store data in non-tabular format.
        - Offer **flexible schema designs, scalability, high performance** for unstructured/semi-structured data.
        - **Schema-Free**, **Horizontal Scalability**.

        - **Diverse Data Models**: Document-Oriented (e.g., MongoDB), Key-Value, Column-Family, Graph.
        - Used for real-time analytics, content management, big data applications.
    - **SQLite**: Lightweight, self-contained, serverless, open-source relational database. Stores database in a single file. Default in Django.

Influence of a Web Browser on Web Development

- Browsers allow non-technical users to interact with the internet visually.
- **Influence**:
    - **Rendering Engine Compatibility**: Developers ensure websites are compatible with various browser engines (Blink, Gecko, WebKit).
    - **CSS and HTML Standards**: Browsers adhere to W3C standards; developers write compliant code.
    - **JavaScript Execution**: JavaScript executed by browser's JS engine; code must work efficiently across implementations.
    - **Feature Detection and Progressive Enhancement**: Leverage modern browser capabilities; provide fallbacks for unsupported features.
    - **Cross-browser Testing**: Used to ensure applications work across different browsers/versions.
    - **Debugging and Optimisation**: Browsers provide tools to improve performance/UX.
- **Developer (Dev) Tools**:
    - Built-in tools in modern web browsers for **debugging, testing, and optimising** webpages.
    - **Features**:
        - **Elements Inspectors**: Examining HTML structure.
        - **Style Editors**: Modifying CSS styles in real-time.
        - **JavaScript Consoles**: Debugging JavaScript code.
        - **Network Monitors (Network Tab)**: Analysing network requests and responses (identifies slow resources, observes caching/file sizes, shows load time, file size/type, HTTP status codes).
        - **Performance Profilers**: Identifying performance bottlenecks.
        - Tools for testing responsiveness, emulating devices, simulating network conditions.

Cascading Style Sheets (CSS) and its Impact

- **CSS**: Responsible for the **appearance and style of a webpage**.
- Separates **content (HTML) from its design**.
- Can be **inline, embedded, or an external stylesheet**.
- Controls colour schemes, typography, spacing, and layout. Supports advanced layout techniques like Flexbox and CSS Grid for responsive designs.
- **Consistency of Appearance**:
    - Ensures a **consistent design language** (uniform styling) across the web application.

    - CSS variables in preprocessors and frameworks like Tailwind CSS help maintain consistency.
- **Flexibility with Browsers or Display Devices (Responsive Design)**:
    - Enables **responsive design**, ensuring web pages display well on any screen size.
    - **Key Techniques**:
        - **Media Queries**: Apply different styles based on screen size.
        - **Fluid Layouts**: Use percentages or `vw/vh` units.
        - **Mobile-First Approach**: Design for mobile devices first.
    - **Layout Models**:
        - **Flexbox**: **One-dimensional layout** system for distributing space and aligning items in a row or column. Ideal for navigation bars, buttons, form elements.
        - **CSS Grid**: Powerful, **two-dimensional layout** system that manages rows and columns simultaneously. Ideal for complex layouts.
    - **Positioning**: Allows precise control over element placement.
- **CSS Maintenance Tools**:
    - **Preprocessors**: Simplify and organise CSS by adding features not native to CSS.
    - **SASS (Syntactically Awesome Style Sheets)**:
        - CSS preprocessor extending CSS capabilities.
        - **Benefits**: **Variables, Nesting, Partials and Import, Mixins, Inheritance**.
        - Compiled into standard CSS.
    - **Tailwind CSS**:
        - A **utility-first framework** providing a large set of utility classes for rapid and consistent styling directly in HTML.
        - **Benefits**: Efficiency, Consistency, Customisation.
        - **Contrast with SASS**: SASS uses traditional CSS with extensions; Tailwind shifts styling to HTML with pre-defined classes. Can be used together.

Version Control

- **Version Control Systems (e.g., Git)**: Manage changes to documents and computer programs.
- Allow **multiple developers to work simultaneously** without overwriting each other's work.
- **Reasons why Version Control is Important**:
    - **Collaboration**: Enables multiple developers to work concurrently, merge changes, resolve conflicts, track contributions.
    - **History Tracking**: Maintains a history of changes, allowing review of previous versions, comparison, and reversion to earlier states.
    - **Backup and Disaster Recovery**: Acts as a backup mechanism.
    - **Branching and Experimentation**: Supports creating separate lines of development (branches) for new features or bug fixes.
    - **Code Reviews and Quality Assurance**: Provides a platform for sharing code changes and conducting peer reviews.

    - **Deployment and Continuous Integration**: Integrates with deployment workflows.
    - **Traceability and Compliance**: Maintains an audit trail of all code changes.

Code Libraries for Front-End Web Development

- **Code libraries and frameworks**: Essential tools that **simplify and speed up coding** by providing pre-written code solutions.
- **Libraries**: Collections of functions or methods for specific purposes that developers can selectively use.
- **Frameworks**: Provide a **structure or blueprint** for developing applications, enforcing coding patterns. Front-end frameworks streamline UI development.
- **Significance of Code Libraries**:
    - **Efficiency**: Reuse existing code components.
    - **Consistency**: Predefined conventions and styles ensure consistency.
    - **Scalability**: Offer structured solutions for building complex applications.
    - **Maintainability**: Promote code organisation and separation of concerns.
    - **Accessibility**: Many libraries prioritise accessibility.
- **Frameworks for Complex Web Applications**:
    - **React**:
        - JavaScript library for dynamic and responsive user interfaces.
        - Emphasises a "**component-based**" architecture (reusable UI modules).
        - **JSX (JavaScript XML)**: Syntax extension for HTML-like code directly in JavaScript.
        - **Props (properties)**: Immutable data passed from parent to child components.
        - **State**: Mutable data managed within a component; changes trigger automatic re-rendering.
        - Often used with **Django** (robust backend) for interactive frontends.
    - **Angular**: (While some Angular concepts like Services, Dependency Injection, Two-Way Data Binding, and Strict Structure are mentioned, the source directly ties them to an `ngClass` example which is an Angular attribute, but describes it as triggering re-rendering like React's `State`. For conciseness and clarity, I am presenting these as general framework concepts or specifically for Angular only if fully clear.)
        - **Services**: Handle business logic, data fetching, or shared functionality.
        - **Dependency Injection (DI)**: Supplies components/services with needed objects.
        - **Two-Way Data Binding**: Seamless communication between view and model.
        - **Strict Structure**: Enforces structure for components, services, and modules.
    - **Django and React together**:
        - **Django (Backend)**: Manages server-side operations, database, API development (Django REST Framework), authentication, business logic, serving data via RESTful APIs.

        - **React (Frontend)**: Manages client-side rendering, user interactions, displaying data from Django APIs.
        - Often use **separate development servers** (decoupling).
- **Template Engines**:
    - Streamline **HTML generation** and make it more maintainable.
    - Combine HTML templates with data to produce **dynamic HTML content**.
    - Allow separation of HTML layout from business logic.
    - **Django Template Language (DTL)**: Django's built-in engine with template tags, filters, and inheritance.
    - **Benefits**: Dynamic content generation, separation of logic/presentation, security (auto-escapes variables), reusable components.
- **Predesigned CSS Classes**:
    - Provided by **CSS frameworks** (e.g., Bootstrap, Tailwind CSS).
    - Offer predesigned components and layouts for **rapid prototyping and building responsive interfaces** without writing custom CSS.
    - **Tailwind CSS**: Utility-first framework for direct styling in HTML.

Open-Source Software (OSS) in Web Development

- **OSS**: Software whose **source code is open for anyone to inspect, modify, and enhance**.
- Has transformed web development by making it **accessible, collaborative, and innovative**.
- Typically hosted on platforms like GitHub, GitLab. Contributions through "pull requests" are common.
- **Impact**: Instrumental due to its **accessibility, community-driven innovation, and cost-effectiveness**.
- Many popular web technologies (e.g., React, Angular) are open source.

Methods to Support and Manage Load Times

- Focuses on ensuring **main content loads quickly** from user perspective.
- **Core Web Vitals**:
    - **Largest Contentful Paint (LCP)**: Time for largest content element to be visible. Measures perceived load speed.
    - **First Input Delay (FID)**: Time for page to respond to first user interaction. Assesses interactivity.
    - **Cumulative Layout Shift (CLS)**: Measures unexpected visual stability (elements changing position).
- **Methods to Support and Manage Load Times**:
    - **Compression**: Gzip and Brotli compress files on server-side (reduces size by up to 70%).
    - **Code Splitting and Lazy Loading**:
        - **Code Splitting**: Breaks JavaScript code into smaller parts that load only when needed.
        - **Lazy Loading**: Loads resources (images, videos) only when they come into view on the user’s screen.

    - **Asynchronous Loading and Deferred Loading of JavaScript**: Prevents JavaScript from blocking HTML rendering.
        - `async` attribute: Script downloads in parallel with HTML parsing; executes immediately.
        - `defer` attribute: Script downloads in parallel; executes after HTML parsing completes.
    - **Content Delivery Network (CDN)**: Network of servers geographically placed to deliver content quickly. Caches content on edge servers, reduces load on origin server.
    - **Image Optimisation**: Compress, use efficient formats (WebP), lazy load.
    - **Reducing Server Response Time**: Database optimisation, reliable hosting, load balancing.
    - **Monitoring and Measuring Performance**: Tools like Google PageSpeed Insights, Lighthouse, WebPageTest.

Web Content Management Systems (CMS)

- **CMS**: Software that helps users **create, manage, and modify content on a website without needing specialised technical knowledge**.
- **How a CMS Functions**:
    - **Content Management Application (CMA)**: Front-end interface where users create/edit content.
    - **Content Delivery Application (CDA)**: Back-end component that takes content from CMA, applies templates, and assembles the public website.
- **Templates in a CMS**: Control layout, structure, and visual appearance.
    - **Themes**: Collections of templates and stylesheets determining overall appearance.
    - **Template Tags**: Fetch and display dynamic content from the database within templates.
- **Content Storage in a CMS**: Typically use databases (e.g., MySQL) to store content. Content and metadata are saved in the database; media files are stored in separate folders.
- **Prevalence**: WordPress is a leading open-source CMS, powering over 40% of websites globally.
- **Benefits of Using a CMS**:
    - **Ease of Use**: Visual editors and templates allow non-technical users to manage websites.
    - **Time Efficiency**: Expedites development with pre-built templates, themes, and plugins.
    - **Scalability**: Adaptable to growing needs with plugins and themes.
    - **Community Support**: Large open-source communities provide continuous updates.
- **Limitations of Using a CMS**:
    - **Resource Intensive**: Can require significant server resources.
    - **Learning Curve for Customisation**: Deeper customisations often require coding knowledge.

- **CMS vs. Traditional Web Development**: CMS offers **code abstraction, lower development costs, and direct content management** via an admin dashboard, while traditional development offers maximum control.

Contribution of Back-End Web Development

- **Back-end development (server-side)**: Creates the **infrastructure for an application's core functionality**.
- Performs essential tasks like **processing data, managing databases, and executing complex logic**.
- **Key Components**: **Web Servers, Databases, Application Logic**.
- **Contribution to Success**:
    - **Data Management**: Critical for storing, retrieving, updating, and deleting data efficiently from databases.
    - **Security**: Implement security measures (authentication, encryption, access control).
    - **Dynamic Content**: Enables dynamic content and personalised experiences by providing data to the front end via APIs.
    - **Scalability**: Ensures the application can handle growing user bases and traffic (e.g., load balancing, caching).
    - **Business Logic**: Processes complex operations, transactions, and data validation.
    - **Multi-Platform Support**: APIs allow a single backend to support various clients (web browsers, mobile apps).
    - **Reliability**: Ensures the server remains reliable and scalable.
- **Server**: A computer that receives messages from client computers over the internet.
- **Back-end frameworks** (e.g., Django) simplify server creation.
- **Package managers** (e.g., NPM for JavaScript, PIP for Python) manage code libraries.
- **Request-Response Cycle**: Fundamental to web applications, where the frontend sends a request, the backend processes it, and sends a response.
- **APIs (Application Programming Interfaces)**:
    - Act as a **middleman**, allowing different software applications to communicate.
    - Define the rules for data requests and responses.
    - Enable clients to interact with the server's business logic and data without direct database access.
    - **Key Functions**: Data exchange (usually JSON/XML), encapsulation and abstraction, integration with third-party services, support for multiple platforms/devices, standardised communication (REST, GraphQL).
    - **API Endpoints**: Specific URL/URI representing a resource or functionality. Use HTTP methods (GET, POST, PUT, PATCH, DELETE), headers, query parameters, and a request body.
- **ORM (Object-Relational Mapping)**:
    - Allows developers to interact with **relational databases using their programming language** (e.g., Python objects for Django ORM) **without writing raw SQL**.
    - Converts database rows into objects and tables into classes.

    - **Simplifies data manipulation**, reduces SQL injection risk, and improves code readability.

Back-End Process to Manage a Web Request

- Involves coordinated steps from HTTP request to response.
- **Request Reception**:
    - Browser (client) sends HTTP request to web server.
    - For static content, server sends file directly.
    - For dynamic content, web server forwards to web application framework (e.g., Django) via an interface like **WSGI**.
- **Web Framework Role (Application Logic)**:
    - The web framework processes the dynamic request.
    - **Routes the request to the corresponding view or controller function**.
    - Interacts with objects, libraries, and databases to handle the request logic.
    - **Django's Main Components for Backend Operations (MVT architecture)**:
        - **Models (Data Layer)**: Define database table structure as Python classes; Django's ORM uses Python code for database interaction.
        - **Views (Business Logic Layer)**: Handle logic behind web requests, bridging models and templates.
        - **Templates (Presentation Layer)**: Render HTML pages dynamically with data from views.
        - **Django ORM**: Abstracts the database layer, mapping tables to Python classes for CRUD operations.
        - **URLs and Routing**: URL dispatcher maps URLs to views.
        - **Forms**: Handles user input and validation.
        - **Middleware**: Functions processing requests and responses for security, session management.
        - **Django Admin**: Built-in interface for managing database data.
        - **Authentication and Authorisation**: Comprehensive user authentication system.
        - **API Integration**: Django REST Framework (DRF) builds/manages APIs.
- **Objects**: Instances of models used to represent and manipulate database records.
- **Libraries**: Used within back-end code to perform specific tasks.

File/Directory Creation and Text Searching (Shell Scripts)

- For creating files/directories and searching text in text files, **Bash (Bourne Again Shell)** is suitable.
- Widely used in Unix-based systems (Linux, macOS).
- **Capabilities**:
    - **File Manipulation and Directory Operations**: Commands like `mkdir` (create directories), `touch` (create files), `mv` (move files), `cp` (copy files), and `rm` (delete files).
    - **Text Processing Capabilities**: Commands like `grep` (search text by patterns), `sed` (text substitution), and `awk` (text processing/reporting).

    - **Integration with System Commands**: Seamlessly combines shell commands with file operations and text processing.
    - **Portability**: Bash scripts are portable across Unix-based systems.
- **Practical Use in Web Development**: Routinely generate file/folder structures for PWA projects; automate tasks like starting a Node.js Express server.

Web-Based Database and SQL Scripts

- **SQL (Structured Query Language)**: Powerful programming language for **managing and manipulating data** in relational databases.
- **SQL Commands/Concepts**:
    - **SELECT**: **Retrieves data** from the database; can specify fields (columns).
    - **GROUP BY**: Groups rows with the same values into summary rows (often with aggregate functions like `COUNT`, `SUM`).
    - **INSERT INTO**: **Adds new records** (rows) into a table.
    - **UPDATE**: **Modifies existing records** in a table.
    - **DELETE FROM**: **Removes records** from a table.
    - **DROP TABLE**: **Deletes an entire table** or database.
    - **Constraints using WHERE keyword**:
        - **WHERE**: Used to **filter records** based on a specified condition.
        - **AND/OR/NOT**: Combine conditions.
        - **ORDER BY**: Sorts results.
        - **LIMIT**: Limits number of results.
        - **LIKE**: Used for **pattern matching** with wildcards (`%`, `_`).
        - **IN**: Specifies multiple possible values for a column.
    - **Table Joins**: Combine rows from two or more tables based on related columns.
        - **INNER JOIN**: Returns rows with **matching values in both tables**.
        - **LEFT (OUTER) JOIN**: Returns all rows from the **left table**, plus matching from the right (non-matches from right are NULL).
        - **RIGHT (OUTER) JOIN**: Returns all rows from the **right table**, plus matching from the left (non-matches from left are NULL).
        - **FULL OUTER JOIN**: Returns all rows from both tables, combining matches and filling non-matching columns with NULLs.

Object-Relational Mapping (ORM) vs. SQL

- Both are methods for working with databases, but have distinct purposes, advantages, and limitations.
- **ORM**: A technique allowing developers to interact with **relational databases using their programming language** (e.g., Python objects) **without writing raw SQL**.
- **Comparison Categories**:
    - **Abstraction**:
        - **ORM**: Abstracts database operations through **object-oriented models**, simplifying interaction.
        - **SQL**: Requires writing **detailed queries**, offering precise control.
    - **Flexibility**:

        - **ORM**: Less flexible for highly complex/optimised queries compared to raw SQL.
        - **SQL**: Offers **maximum flexibility** and fine-grained control.
    - **Learning Curve**:
        - **ORM**: Can have a learning curve.
        - **SQL**: Syntax is generally more straightforward and widely used.
    - **Code Readability and Maintenance**:
        - **ORM**: Integrates database operations within the programming language, making code **more readable and consistent**.
        - **SQL**: Directly writing SQL can become complex and hard to maintain when queries are scattered.
    - **Security**:
        - **ORM**: Most ORMs automatically **sanitise input**, reducing risk of **SQL injection**.
        - **SQL**: Raw SQL requires careful sanitisation to prevent SQL injection.
    - **Performance**:
        - **ORM**: Can sometimes generate less efficient queries for complex scenarios.
        - **SQL**: Allows highly optimised, database-specific queries.
    - **Database Portability**:
        - **ORM**: Often supports **multiple database systems** with minimal code changes.
        - **SQL**: Queries might need modifications for different database systems.

Collaborative Web Development

- **Collaboration** between front-end and back-end developers is crucial for building functional, efficient, user-friendly, and visually appealing applications.
- **Collaborative Work Practices**: Cross-functional team meetings, daily stand-up meetings, and feedback loops ensure alignment.
- **Interconnectivity and Workflow**:
    - **UX to UI**: UX team provides wireframes; UI designers enhance with visuals.
    - **UI to Front-End**: UI passes prototypes; front-end transforms into interactive code (HTML, CSS, JavaScript).
    - **Version Control with Git**: Allows multiple developers to work simultaneously without overwriting. Branches enable independent feature development and later merging. Facilitates code reviews.
    - **Application Programming Interfaces (APIs)**: The "**glue**" between front end and back end. Enable the front end to request data from the back end and display it dynamically. Testing APIs (e.g., with Postman) is essential.
    - **Cross-functional Collaboration**: Leveraging strengths, sharing knowledge, solving complex problems.
    - **Iterative Development**: Facilitates incremental building, testing, and refining features.

Progressive Web Applications (PWAs) Design Considerations

- **PWA**: Built using **web platform technologies**.
- Provides user experience like a platform-specific app.
- Can run on **multiple platforms/devices from a single codebase**, be installed, operate offline/in background, and integrate with devices.
- **Technical features**: At least one HTML page, loads CSS and JavaScript, accesses Web APIs.
- **Additional features**: Web app manifest file (installation info), service worker (basic offline experience).
- **PWA Design Considerations**: Engage users (e.g., via push notifications).
- **Application of Design and UI/UX Principles**:
    - **User Experience (UX)**: Focuses on understanding and improving how users interact with a website/application, prioritising functionality, usability, and the overall user journey.
    - **User Interface (UI)**: Focuses on visual presentation and interactive elements.
    - **Principles applied in PWA design**:
        - **Font**: Selecting readable fonts and ensuring consistent typography.
        - **Colour**: Defining colour schemes for visual appeal and brand alignment.
        - **Audio/Video**: Consider integration and optimisation for various network conditions (data compression and encoding).
        - **Navigation**: Designing intuitive navigational elements (menus, links, buttons).
        - **Accessibility**: Ensuring websites and applications are usable by everyone, including individuals with disabilities. W3C's **Web Content Accessibility Guidelines (WCAG)** provide the framework. Considerations include **alt text for images** and ensuring content is perceivable, operable, understandable, and robust across assistive technologies.
        - **Content Security Policy (CSP)**: Mentioned as a web security concept for XSS mitigation, implying its application in PWAs.

SSA

- **Designing Software**
    - **Benefits of developing secure software**
        - Essential for **protecting data**, **minimising vulnerabilities**, **preventing cyber attacks**.
        - Safeguards sensitive information from **unauthorised access, manipulation, theft**.
        - **Data Protection**
            - **Encryption**: Data at rest/in transit.
            - **Access Controls**: Role-based access control (RBAC).
        - **Confidentiality**: Data private, protected from unauthorised access.
        - **Integrity**: Prevents unauthorised data modification/tampering.
        - **Availability**: Data/services accessible to legitimate users.
        - **Compliance with regulations**: GDPR, HIPAA, PCI DSS.
        - **Customer trust**: Enhances loyalty and satisfaction.
        - **Business continuity**: Reduces data loss/service disruptions.
        - **Minimising cyber attacks & vulnerabilities**: Secure coding, patching.
        - **Prevent data breaches**: Protects sensitive info.
        - **Mitigate against cyber threats**: Reduces malware, ransomware, phishing, DoS.
        - **Cost savings**: Addressing security early is more cost-effective.
        - **Competitive advantage**: Differentiates organisations.
        - **Vulnerabilities**: Weaknesses exploited (e.g., SQL Injection, XSS). Avoided by **input validation, parameterised queries, output encoding**.
- **Fundamental software development steps (SDLC)**
    - Security must be **embedded from the beginning**.
    - Phases: **Requirements, Specifications, Design, Development, Integration, Testing & Debugging, Installation, Maintenance**.
    - **Requirements definition**: Focus on **non-functional security requirements** (e.g., password hashing).
    - **Determining specifications**: Translate requirements into actionable details (e.g., **strong password hashing, rate limiting, HTTPS**).
    - **Design**: Define architecture & components.
        - **Key design principles**: **Modularity, Abstraction, Encapsulation, Reusability, Maintainability**, **MVC Pattern** (Model, View, Controller).
    - **Development**: Code written per design.
    - **Integration**: Modules tested together; security assessment of existing systems, secure APIs/databases.
    - **Testing and Debugging**: Identify/fix vulnerabilities (e.g., SQL injection).
    - **Installation**: Secure process. Key practices: **Secure Connections (HTTPS), Environment Isolation, Database Configuration, Access Control/Permissions**.

    - **Maintenance**: Regular updates, security patches, continuous monitoring.
- **How end-user capabilities/experience influence secure design features**
    - **User-centered design (UCD)**: Puts end-user first, minimises security risks from misuse.
    - **UCD principles for secure software**: **Simplicity, Clarity, Transparency**.
    - **Common user behaviours affecting security & solutions**:
        - **Weak password practices**: Implement strong policies.
        - **Phishing/social engineering**: User education.
        - **Failure to update software**: Encourage automatic updates.
        - **Bypassing security protocols**: Design user-friendly protocols.
    - **Influence on design features**: **User-friendly authentication** (e.g., MFA options), **Role-Based Access Control**, **Training and awareness**, **User feedback/testing**, **Simplified security controls**.
- **Fundamental software design security concepts**
    - **CIA Triad**:
        - **Confidentiality**: Data private. Ensure via **encryption, HTTPS**.
        - **Integrity**: Data accurate, consistent, trustworthy. Ensure via **checksums, digital signatures, hash functions**.
        - **Availability**: Services accessible. Ensure via **redundancy, load balancing, disaster recovery**.
    - **AAA**:
        - **Authentication**: Verifies user identity. Ensure via **strong mechanisms (passwords, 2FA, biometrics)**.
        - **Authorisation**: Enforces access control based on roles. Ensure via **RBAC/ABAC**.
        - **Accountability**: Traces actions to individual. Ensure via **logging, auditing**.
- **Security features incorporated into software**
    - **Data protection**: **Secure password hashing (Bcrypt), data encryption (TLS)**.
    - **Security measures**: **Input validation/sanitisation** (prevent SQLi/XSS), **parameterised queries**, secure coding practices.
    - **Privacy protection**: Clear privacy policy, user consent, data anonymisation.
    - **Regulatory compliance**: Align with laws (e.g., Privacy Act 1988), regular audits.
    - **User authentication & authorisation**: **MFA**, access controls, monitoring user access.
- **Contribution of Cryptography & Sandboxing to 'Security by Design' (SbD)**
    - **Security by Design (SbD)**: Integrates security controls **from design phase**, not an afterthought.
    - **Cryptography**: Encodes info. Ensures **confidentiality, integrity, authenticity**. Used for **data encryption, password hashing**.
    - **Sandboxing**: Isolated environment for safe code testing. Limits breach impact. E.g., web browser sandboxing, isolating third-party APIs.

- **‘Privacy by Design’ (PbD) approach**
    - Proactive approach, integrates privacy from outset.
    - **Proactive not reactive**: Anticipates issues during design.
    - **Embed privacy into design**: Protections built-in by default (data minimisation, anonymising).
    - **Respect for user privacy**: User interests paramount, user control over data.
- **Testing and evaluating software security and resilience**
    - **Vulnerabilities**: Weaknesses attackers exploit.
    - **Resilience**: Ability to withstand and recover from failures/attacks.
    - **Determining vulnerabilities**: Identify weaknesses via tools, inspection, simulated attacks.
    - **Hardening systems**: Adopting best practices.
    - **Handling breaches**: **Incident response plan** (detect, contain, mitigate).
    - **Maintaining business continuity**: Critical operations during/after disruption.
    - **Conducting disaster recovery (DR)**: Restoring systems after major failure/attack. Involves RTO, RPO.
- **Strategies used by software developers to manage security**
    - **Code review**: Systematic examination to identify bugs, improve quality, ensure secure coding.
    - **Static Application Security Testing (SAST)**: Analyses source code *without executing* to find vulnerabilities early (e.g., SQLi, unvalidated input). **Advantages**: Early detection, immediate feedback.
    - **Dynamic Application Security Testing (DAST)**: Tests running application by simulating attacks (e.g., XSS, SQLi, weak session management). **Advantages**: Finds runtime vulnerabilities, realistic assessment.
    - **Vulnerability assessment**: Scans to identify, quantify, prioritise weaknesses.
    - **Penetration testing (Pen testing)**: Proactive simulation of real-world attacks to exploit vulnerabilities. Assesses system resilience.
- **Defensive data input handling practices**
    - **Input validation**: Verifies user input format/constraints.
        - Types: Client-side, **Server-side (critical for security)**.
        - Best Practice: **Never trust user input, use whitelist validation**.
    - **Sanitisation**: Cleans/escapes harmful characters/scripts (e.g., for XSS, SQLi).
    - **Error handling**: Manages errors gracefully.
        - Key Principles: **Do not expose sensitive information**, **graceful error recovery**, **log errors securely**.
- **Safe Application Programming Interface (API) design & implementation**
    - **Common API vulnerabilities**: Lack of Auth/Auth, Data Exposure, Insecure Communication, Injection Attacks, Lack of Rate Limiting.
    - **Best Practices for Securing APIs**: **Authentication/Authorisation (OAuth 2.0), HTTPS, Input Validation/Sanitisation, Limit Data Exposure, Rate Limiting, API Keys/Tokens, Audit/Monitor API Usage**.

- **Code considering efficient execution for the user**
    - **Memory management**: Wise resource use, avoids leaks, protects sensitive data in memory.
    - **Session management**: Tracks user interactions, authenticates.
        - Concepts: **Session IDs** (unique, random), **Session Timeout**, **Session Regeneration**, **Secure Cookies** (HttpOnly, Secure).
        - Vulnerabilities: **Session Hijacking, Session Fixation**.
    - **Exception management**: Handles unexpected problems gracefully, prevents crashes, protects from threats.
- **Secure code to minimise vulnerabilities in user action controls**
    - **Broken authentication & session management**: Allows unauthorised access.
        - Prevention: **Strong password policies, MFA, secure session management**.
    - **Cross-Site Scripting (XSS)**: Injecting malicious scripts into web pages.
        - Prevention: **Sanitise input, Output encoding**.
    - **Cross-Site Request Forgery (CSRF)**: Attacker tricks user into unintended actions.
        - Prevention: **CSRF Tokens, SameSite Cookies**.
    - **Invalid forwarding and redirecting**: Information not found in source.
    - **Race conditions**: Concurrent access to shared resources leads to unpredictable outcomes.
        - Prevention: Use **locks**.
- **Secure code to protect user file & hardware vulnerabilities (File Attacks & Side Channel Attacks)**
    - **File attacks**: Improper handling of uploads, permissions.
        - Vulnerabilities: Malicious File Uploads, Path Traversal, Improper File Permissions, File Name Injection.
        - Prevention: **File Type Validation, File Name Sanitisation, Limit File Permissions, Random File Names, Secure Directories, Access Control, Malware Scanning, Size Limits, Encryption**.
    - **Side channel attacks**: Gather info from physical implementation, not direct software flaws (e.g., timing, power, cache).
        - Mitigation: **Constant-Time Algorithms, Blinding Techniques, Random Delays, Shielding, Proper Cache Management, Hardware Isolation (TEE), Firmware Updates, Secure Boot, Physical Security**.
- **Impact of safe and secure software development**
    - **Benefits of collaboration**: Diverse points of view, task delegation by expertise, improved solution quality.
    - **Benefits to an enterprise**: Improved products/services, influence on future development, improved work practices, productivity, business interactivity (enables remote work).
    - **Social, ethical, and legal issues**:

- **Employment**: Job creation (cybersecurity) vs. displacement (AI/automation).
- **Data security**: Safeguarding personal data (identity theft prevention).
- **Privacy**: Protecting personal info (consent, GDPR).
- **Copyright**: Legal rights to original works (software code).
- **Intellectual property (IP)**: Protecting creations (patents, trademarks, licensing).
- **Digital disruption**: Impact on business models, potential digital divide.

Program for Web

Here are very concise, indented dot-point notes on the provided source:

- **Programming for the Web**
    - **Applications of Web Programming**
        - **Interactive Website/Webpages**: Enables **dynamic functionality** for browsing, communication, and interaction online.
            - Examples: Social media, online learning, web-based games, project management, government/public service.
            - Focus: User engagement, data management, productivity.
        - **E-commerce**: Facilitates **buying and selling goods/services** over the internet.
            - Examples: Amazon, eBay, Alibaba.
            - Key components: Internet access, mobile tech, secure payment gateways, UX innovations (recommendations, reviews).
        - **Progressive Web Apps (PWAs)**: Combines **web and mobile app features**.
            - **Installable** on devices without app stores.
            - **Faster, more reliable**, and offer **offline functionality**.
            - Built with **HTML, CSS, JavaScript, Web APIs**.
            - **Adapt to screen sizes** and **stay up-to-date**.
            - Can offer **push notifications** and **full-screen mode**.
            - **Recognition**: Installation prompt, home screen icon, full-screen, push notifications, offline, responsive design.
            - Built with **progressive enhancements**.
- **Data Transfer on the Internet**
    - **Data Packets**: Data (e.g., email, video) is broken into **smaller units** for efficient, reliable, scalable transmission.
        - Each packet contains **part of data + additional info** (e.g., for reassembly).
        - **Components**:
            - **Header**: **Metadata** (Source/Destination IP, Sequence Number, Protocol Info, Error Checking).
            - **Payload**: The **actual data**.
            - **Trailer**: Indicates **end of packet** (optional error-checking).
        - **Packet Switching**: Packets **take multiple routes**; routers direct them, enabling **dynamic routing**.
            - Fundamental for efficient, reliable internet communication.
            - **Key Features**: Data division (typically 1,500 bytes), independent routing, reassembly (sequence numbers), error detection/retransmission, shared network resources.
            - **Advantages**: Efficiency, reliability, error recovery, scalability.
            - **Disadvantages**: Variable delays, potential for packet loss.

        - **Applications**: Internet, Cellular Networks (4G/5G), Email Systems.
    - **Internet Protocol (IP) Addresses**
        - **IP**: Principal **communication protocol for relaying data packets** across networks.
        - **IP address**: Unique **postal address for a device** on the internet.
        - **IPv4**: **32-bit address**, ~4.29 billion unique addresses (e.g., 192.168.1.1). Running out.
        - **IPv6**: **128-bit address**, 340 undecillion addresses (e.g., 2001:0db8:...).
        - **Key Differences (IPv4 vs. IPv6)**: Format (decimal vs. hexadecimal), address count (limited vs. unlimited), IPv6 has built-in security, better mobile support.
    - **Domain Name System (DNS)**
        - **DNS**: Translates **user-friendly domain names** (e.g., "www.example.com") into **IP addresses** for computers. Acts as internet's "address book".
        - **How it Works**:
            - User enters domain.
            - Browser contacts **DNS resolver** (often ISP).
            - Resolver checks **DNS cache**.
            - If not cached, resolver queries **hierarchy of DNS servers** (root, TLD, authoritative) to get IP.
            - IP returned to browser, enabling connection.
        - **Common DNS Records**:
            - **A record**: Maps domain to **IPv4**.
            - **AAAA record**: Maps domain to **IPv6**.
            - **CNAME record**: Maps domain to **another domain**.
            - **MX record**: Specifies **mail servers**.
        - **Port**: DNS typically uses **Port 53**.
- **Web Protocols and Their Ports**
    - **Importance**: Facilitate internet communication, operate on designated ports, crucial for software engineers to design efficient, reliable, secure applications.
    - **HTTP (Hypertext Transfer Protocol)**: Allows browsers to **fetch resources** (e.g., HTML pages) from web servers.
        - **Stateless Protocol**: Each request is independent; server doesn't remember previous requests.
        - **Unsecured**: Data can be intercepted.
        - **Port**: Typically uses **Port 80**.
    - **HTTPS (Hypertext Transfer Protocol Secure)**: **Secure version of HTTP**.
        - Uses **SSL/TLS encryption** for secure data transmission.
        - **Crucial for sensitive info** (login, payment).
        - **Security Indicator**: Padlock icon in browser.
        - **Port**: Commonly uses **Port 443**.
    - **TCP/IP (Transmission Control Protocol/Internet Protocol)**: **Foundational communication protocol suite** for the internet.

        - **TCP**: Responsible for **reliable data transmission**; breaks data into packets, reassembles them, establishes connection (connection-oriented). Example: file downloads.
        - **IP**: Handles **addressing and routing** data packets to destination.
        - **Difference**: IP defines address, TCP transports data reliably to that address.
        - **Transport Protocol**: Web servers primarily use **TCP** for reliable, ordered, error-checked delivery.
    - **FTP (File Transfer Protocol)**: Used for **transferring files** between client and server (uploading, downloading, managing).
        - **Ports**: Uses **Ports 20** (data) and **21** (control).
    - **SFTP (Secure File Transfer Protocol)**: **Secure version of FTP**, encrypted over SSH.
        - **Port**: Uses **Port 22** (same as SSH).
        - **Key Point**: Preferred over FTP for sensitive data due to confidentiality and integrity.
    - **SSL (Secure Sockets Layer) / TLS (Transport Layer Security)**: **Encryption protocols** for internet communications.
        - **TLS is successor to SSL**.
        - Uses **cryptographic algorithms** to scramble data.
        - **Authenticates server's identity** (prevents man-in-the-middle attacks).
        - **No specific ports**, but used with protocols like HTTPS (443), SMTP (25, 587), IMAP (993).
    - **SMTP (Simple Mail Transfer Protocol)**: Used for **sending email messages** between servers.
        - **Port**: Default is **Port 25**.
    - **POP3 (Post Office Protocol version 3)**: Email retrieval protocol; **downloads emails** to device, often removing from server.
        - **Port**: Uses **Port 110**.
    - **IMAP (Internet Message Access Protocol)**: Email retrieval protocol; allows **viewing emails remotely** on server, synchronising across devices.
        - **Port**: Uses **Port 143**.
    - **Secure Versions (email)**: POP3S (995), IMAPS (993) use SSL/TLS.
- **Securing the Web**

    - Ensures **confidentiality, integrity, availability** of data/systems. Often legally required (e.g., GDPR).
    - **SSL Certificates (now TLS)**: **Digitally bind a cryptographic key** to an organisation's details, enabling **secure HTTPS connections**.
        - **Digital certificate** that authenticates website's identity.
        - Issued by **Certificate Authority (CA)**.
        - Contains domain, organisation, CA, public key, expiration.
        - **How they work**: Browser requests secure connection, server sends certificate, browser verifies validity, handshake generates **session key**, then **encrypted communication**.
        - Ensure secure browsing (encryption), authentication, data integrity.

- **Encryption Algorithms**: Convert **readable data (plaintext) into unreadable (ciphertext)**.
  - **Symmetric Encryption**: Uses **same key** for encryption/decryption. Example: **AES**.
  - **Asymmetric Encryption**: Uses **two keys (public for encryption, private for decryption)**. Example: **RSA**. Used in SSL/TLS for key exchange.
- **Encryption Keys**: Sequences of characters determining algorithm output; must be kept secure.
  - **Public Key**: Shared openly to encrypt.
  - **Private Key**: Kept secret to decrypt.
  - Longer keys (e.g., 128-bit, 256-bit) provide stronger security.
  - **Key Exchange**: Securely sharing keys; SSL/TLS uses asymmetric for exchange, symmetric for data transfer. Diffie-Hellman algorithm.
- **Plaintext and Ciphertext**:
  - **Plaintext**: **Original, readable** data.
  - **Ciphertext**: **Unreadable format** after encryption.
- **Authentication and Authorisation**:
  - **Authentication**: **Verifies who a user is** (e.g., username/password, biometrics, 2FA).
  - **Authorisation**: **Determines what an authenticated user can access** (permissions).
  - Together, prevent unauthorised access.
  - **OAuth**: Authorisation framework for apps to access resources without sharing passwords, uses access tokens.
  - **OpenID Connect (OIDC)**: Builds on OAuth for authentication, used for Single Sign-On (SSO).
- **Hash Values**: Fixed-size string output from input data via a **hash function**.
  - **Properties**: Deterministic, Irreversible, Collision-Resistant.
  - **Uses**: Verifying data integrity (file checksums), securely storing passwords (store hashed passwords).
  - Algorithms: SHA-256, MD5.
- **Digital Signatures**: Verify **authenticity and integrity** of message/document.
  - Created using sender's **private key**, verified with sender's **public key**.
  - Provide: Authentication, Integrity, Non-Repudiation.

- **Effect of Big Data on Web Architecture**

  - **Big Data**: **Massive datasets** (Volume, Variety, Velocity, Veracity) that traditional methods can't manage.
  - **Four Vs**:
    - **Volume**: Amount of data (terabytes, petabytes). Challenges: Storage, processing.
    - **Variety**: Types/formats (structured, semi-structured, unstructured). Challenges: Integration, analysis.
    - **Velocity**: Speed of generation/processing (real-time). Demands real-time processing systems.
    - **Veracity**: Quality, trustworthiness, reliability. Challenges: Ensuring accuracy (filtering, cleaning).

    - **Opportunities**: Customer personalisation, predictive analytics, enhanced decision making.
    - **Challenges**: Storage/processing (distributed computing, cloud), data integration/analysis (ML, data mining), real-time processing.
    - **Effect on Web Architecture**: Driven evolution to more **scalable, flexible, real-time** systems.
    - **Data Mining**: Discovering **patterns, trends, insights** from large datasets.
        - **Applications**: Describing target data, predicting outcomes, fraud detection, user learning, bottleneck detection.
        - **Techniques in Streaming**:
            - **Association**: Recommends items consumed together (e.g., Spotify song sequences).
            - **Classification**: Categorises users (e.g., "hip-hop fans").
            - **Clustering**: Groups users by shared habits without predefined categories (e.g., mood playlists).
            - **Predictive Analytics**: Anticipates user behavior (e.g., Discover Weekly).
        - **Benefits (Streaming)**: Enhanced personalisation, optimised content delivery, business intelligence.
        - **Challenges (Streaming)**: Data privacy, scalability, real-time analysis.
    - **Metadata**: "**Data about data**".
        - **Purposes**: Descriptive (title, author), Structural (organisation), Administrative (access rights).
        - **Relationship with Big Data**: Manages overwhelming info; helps organise/retrieve user activity, preferences efficiently.
        - **Effect on Web Apps**: Enables performance/scalability, data processing/personalisation, search/filtering, security/privacy concerns.
        - **Role in Web Architecture**: Data organisation, enhanced searchability, data governance/compliance.
    - **Streaming Service Management**: Real-time data processing for immediate responses (e.g., financial transactions, social media feeds).
        - **How Streaming Services Manage Big Data**:
            - **Data Storage/Retrieval**: Distributed server networks (CDNs) for low latency.
            - **Real-Time Data Processing**: Caching, buffering, CDNs for seamless experience.
            - **Data Compression/Encoding**: Reduce file sizes (codecs: H.264/H.265, AAC). Adaptive bitrate streaming.
            - **Personalisation/Recommendations**: ML algorithms, data mining (collaborative, content-based filtering).
- **Designing Web Applications**

    - **World Wide Web Consortium (W3C)**: International organisation developing **open standards** for web's long-term growth.
        - **Standards** cover HTML, CSS, XML, SVG, Semantic Web, ensuring accessibility, interoperability, security.

        - **Web Accessibility Initiative (WAI)**: Makes web accessible to individuals with disabilities. **Web Content Accessibility Guidelines (WCAG)**.
        - **Internationalisation (I18n)**: Standards for multilingual content, diverse cultural conventions in PWAs.
        - **Web Security**: Standards (HTTPS, CSP, CORS, secure authentication) for secure PWAs, mitigating XSS, CSRF, injection attacks.
        - **Privacy**: Guidelines for handling sensitive PWA data (user preferences, location), user consent.
        - **Machine-Readable Data**: Standards (XML, SVG) for representing/exchanging data, enhancing interoperability, search engine visibility.
- **Web Development System Elements**
    - Web app structure: **Front-end (client-side) and Back-end (server-side)**.
    - **Interaction**: **Client Request → Server Processing → Server Response → Browser Renders**.
        - **Client Request**: User initiates (URL, link), browser sends HTTP request (resource, headers). Can be GET (retrieve) or POST (send data).
        - **Server Processing**: Web server receives, processes request. For dynamic pages, runs server-side code (Python, PHP, Node.js), interacts with database.
        - **Server Response**: Server sends HTTP response (status code, data e.g., HTML/JSON, headers).
        - **Browser Renders**: Parses HTML (DOM), loads resources (CSS, JS, images), executes client-side scripts.
    - **Code**:
        - **Static Code**: Hard-coded HTML, **unchanging**.
        - **Dynamic Code**: Generated from database/CMS by server-side languages, **changes based on conditions**.
    - **Delivery**:
        - **Static Delivery**: Page pre-generated, stored on server/CDN, **sent directly to browser (fast)**.
        - **Dynamic Delivery**: Server generates page **on-the-fly** in response to requests (for external/user-specific data).
    - **Client Browser Interaction**:
        - **Static Client Browser**: Content same for all users, no interaction beyond viewing.
        - **Dynamic Client Browser**: Uses **JavaScript** for real-time changes based on user interactions or asynchronous data fetching (AJAX).
    - **Client-Side (Front-End) Web Programming**: Code runs on **user's device** (web browser). Creates **interactive parts** users see.
        - **Languages/Technologies**:
            - **HTML**: Structures information.
            - **CSS**: Styles webpage (layout, colours, fonts).

            - **JavaScript**: Primary language for client-side, makes pages interactive. Manipulates DOM, fetches data asynchronously (AJAX), validates input.
        - **Less secure** (visible to users), **fast for small tasks** (no server communication).
    - **Server-Side (Back-End) Web Programming**: Handles operations on a **web server**.
        - Processes requests requiring database interaction, manages databases, security, file operations.
        - **Languages/Technologies**: Python (Django, Flask), Node.js, PHP, Ruby (Rails), Java (Spring Boot).
        - **More secure** (hidden from users), generally **slower** (server-client communication).
    - **Interfacing with Databases (SQL/NoSQL)**
        - **Database**: Structured collection of data, easily accessed/managed. Web server interacts via language like SQL.
        - **Relational Databases (SQL)**: Tabular format (tables, rows, columns).
            - Data points have **unique IDs (keys)**. Relationships via primary/foreign keys.
            - Ideal for **structured data**. Use **SQL queries**.
            - Follow **ACID principles** (Atomicity, Consistency, Isolation, Durability) for reliability.
            - Require **schema defined upfront**.
            - Examples: MySQL, PostgreSQL, SQLite, MS SQL Server, Oracle, MariaDB, Amazon Aurora.
            - **Entities**: Real-world objects (e.g., Car, Course) modeled as classes/tables.
            - **Attributes**: Properties (fields) of an entity.
            - **Instances**: Actual data records (rows).
            - **Entity Relationships**: One-to-One, One-to-Many (ForeignKey), Many-to-Many (ManyToManyField).
        - **NoSQL Databases**: **Non-relational**, non-tabular format.
            - **Flexible schema**, **scalability**, high performance for **unstructured/semi-structured data**.
            - **Schema-Free**, **Horizontal Scalability**.
            - **Diverse Data Models**: Document-Oriented (MongoDB), Key-Value (Redis), Column-Family (Cassandra), Graph (Neo4j).
            - Used for real-time analytics, content management, big data.
            - Example: MongoDB (flexible JSON-like documents).
        - **SQLite**: Lightweight, self-contained, serverless, relational database. Stores database in single file. Default in Django.
        - **Managing Data**: Django admin interface or SQLite shell (SQL commands/Django ORM).
- **Influence of Web Browser on Web Development**
    - Browsers enable **visual interaction** for non-technical users.
    - **Influence**:

        - **Rendering Engine Compatibility**: Developers ensure consistent rendering across engines (Blink, Gecko, WebKit).
        - **CSS/HTML Standards**: Browsers adhere to W3C standards; developers write compliant code.
        - **JavaScript Execution**: JavaScript engine execution; developers write efficient, cross-browser JS.
        - **Feature Detection/Progressive Enhancement**: Leverage modern capabilities, provide fallbacks for unsupported features.
        - **Cross-browser Testing**: Essential for ensuring apps work across different browsers/versions.
        - **Debugging/Optimisation**: Browsers provide tools.
    - **Developer (Dev) Tools**: Built-in browser tools for debugging, testing, optimising.
        - **Features**: Elements Inspectors (HTML), Style Editors (CSS), JavaScript Consoles (JS debugging), **Network Monitors (Network Tab)**.
        - **Network Monitor**: Analyses network requests/responses, identifies slow resources, observes caching, file sizes. Key metrics: Load Time, File Size/Type, HTTP Status Codes.
        - Performance Profilers, tools for responsiveness testing, device/network simulation.
- **Cascading Style Sheets (CSS)**
    - Controls **appearance and style** of webpage, **separates content (HTML) from design**.
    - Controls colour schemes, typography, spacing, layout (UI). Supports **responsive designs** (Flexbox, CSS Grid). Creates transitions/animations.
    - **Consistency of Appearance**: Ensures **uniform styling** across application. CSS variables, frameworks (Tailwind CSS) aid consistency.
    - **Flexibility with Browsers/Display Devices**: Enables **responsive design** (display well on any screen size).
        - **Key Techniques**:
            - **Media Queries**: Apply styles based on **screen size**.
            - **Fluid Layouts**: Use percentages/vw/vh units for widths/heights.
            - **Mobile-First Approach**: Design for mobile first.
        - **Layout Models**:
            - **Flexbox**: **One-dimensional** system for aligning items in row/column. Ideal for nav bars, buttons.
            - **CSS Grid**: Powerful, **two-dimensional** system for rows/columns simultaneously. Ideal for complex layouts (galleries, dashboards).
        - **Positioning**: Precise control over element placement (static, relative, absolute, fixed).
    - **CSS Maintenance Tools**: Simplify and organise CSS, streamline styling.
        - **SASS (Syntactically Awesome Style Sheets)**: CSS preprocessor, extends CSS capabilities.

        - **Benefits**: Variables (reusable values), Nesting (logical grouping), Partials/Import (split files), Mixins (reusable styles), Inheritance (@extend).
        - **Compiling**: Needs a SASS compiler (Node-sass, Dart-sass) to convert to standard CSS.
    - **Tailwind CSS**: **Utility-first framework**. Provides **utility classes for rapid styling directly in HTML** (e.g., bg-blue-500).
        - **Benefits**: Efficiency, Consistency, Customisation.
        - **Contrast with SASS**: SASS extends CSS; Tailwind shifts styling to HTML with pre-defined classes.
- **Version Control**
    - Systems (like Git) **manage changes** to code, allowing multiple developers to work simultaneously without overwriting.
    - **Reasons for Importance**:
        - **Collaboration**: Concurrent work, merging changes, conflict resolution.
        - **History Tracking**: Review previous versions, compare changes, revert to earlier states (debugging, auditing).
        - **Backup/Disaster Recovery**: Protects against data loss.
        - **Branching/Experimentation**: Separate development lines for features/bug fixes.
        - **Code Reviews/Quality Assurance**: Platform for sharing code and peer reviews.
        - **Deployment/Continuous Integration**: Integrates with automated processes.
        - **Traceability/Compliance**: Audit trail of code changes.
- **Code Libraries for Front-End Web Development**
    - **Libraries**: Collections of **functions/methods for specific purposes** that developers selectively use.
    - **Frameworks**: Provide a **structure/blueprint** for developing applications, enforce coding patterns. Front-end frameworks streamline UI development.
    - **Significance**: Efficiency (reuse code), Consistency (predefined conventions), Scalability (structured solutions), Maintainability (organisation), Accessibility.
    - **Frameworks for Complex Web Applications**:
        - **React**: JavaScript library (often considered framework) for **dynamic, responsive user interfaces**. Emphasises **component-based architecture** (reusable UI modules).
            - **JSX (JavaScript XML)**: HTML-like code in JS files.
            - **Props**: Immutable data passed parent-to-child.
            - **State**: Mutable data within a component; changes trigger re-rendering.
            - **Services**: Handle business logic, data fetching.
            - **Dependency Injection (DI)**: Supplies components/services with needed objects.
            - **Two-Way Data Binding**: Seamless communication between view and model.

            - **Strict Structure**: Enforces structure for scalability/maintainability.
            - **Django and React Together**: Django (backend) + React (frontend).
                - **Django**: Server-side operations, database, API development (REST Framework), authentication, business logic.
                - **React**: Client-side rendering, user interactions, displaying data from Django APIs, state management. Fetches data using HTTP (Axios, Fetch API).
                - **Separate Servers**: React dev server (e.g., localhost:3000) for hot-reloading.
                - **Integration in Production**: Build React app and serve static files with Django, or deploy separately.
                - **Benefits**: Dynamic UI, reusability, modern practices, flexibility, "best of both worlds".
    - **Template Engines**: Streamline **HTML generation**, combine HTML templates with data to produce **dynamic HTML**. Separates HTML layout from business logic.
        - **Django Template Language (DTL)**: Django's built-in engine.
        - **Inheritance**: `{% block %}` and `{% extends %}` for base/child templates (consistency, reusability).
        - **Benefits**: Dynamic content, separation of logic/presentation, security (auto-escapes XSS), reusable components.
        - **Context Processors**: Provide extra context data globally across templates (e.g., user info).
        - Other engines: Handlebars, Mustache, Pug (client-side).
    - **Predesigned CSS Classes**: Provided by **CSS frameworks** (Bootstrap, Foundation, Bulma).
        - Offer predesigned components/layouts for rapid prototyping.
        - Example: **Tailwind CSS** (utility-first framework) provides specific, reusable classes for direct HTML styling.
- **Open-Source Software (OSS)**
    - Source code is **open for inspection, modification, enhancement**. Hosted on GitHub, GitLab, Bitbucket.
    - **Benefits**: Accessibility, community-driven innovation, cost-effectiveness.
    - **Impact on Web Development**: Instrumental due to accessibility, innovation, cost-effectiveness. Many popular web technologies are open source (Bootstrap, React, Angular, jQuery).
    - **Famous Projects**: Linux, Apache HTTP Server, WordPress, React, Angular, VS Code.
- **Support and Manage Load Times**
    - **Web Performance Metrics (Core Web Vitals)**:
        - **Largest Contentful Paint (LCP)**: Measures **main content loading performance**. Good: ≤ 2.5s.

        - Improvements: Optimise images/videos (compress, WebP, lazy-load), CDN, minimise render-blocking resources, Server-Side Rendering (SSR).
    - **First Input Delay (FID)**: Time for page to **respond to first user interaction**. Good: ≤ 100ms.
        - Improvements: Reduce JavaScript execution (minify, defer), split long tasks, optimise third-party scripts, Web Workers.
    - **Cumulative Layout Shift (CLS)**: Measures **unexpected visual stability** (elements shifting position). Good: ≤ 0.1.
        - Improvements: Specify dimensions for images/videos, reserve space for dynamic content.
- **Methods to Support/Manage Load Times**:
    - **Compression**: Gzip, Brotli compress files server-side.
    - **Code Splitting**: Breaks JavaScript into smaller parts, loads when needed.
    - **Lazy Loading**: Loads resources (images, videos) only when in view. `loading="lazy"` attribute.
    - **Asynchronous/Deferred Loading of JavaScript**: Prevents JS from blocking HTML rendering.
        - `async`: Downloads in parallel, executes immediately.
        - `defer`: Downloads in parallel, executes **after HTML parsing complete**.
    - **CDN Caching**: Stores content on edge servers worldwide, reduces load, improves speed. **Content Delivery Network (CDN)** distributes content globally. Example: Cloudflare.
    - **Image Optimisation**: Compress, use efficient formats (WebP), Lazy Loading.
    - **Reducing Server Response Time**: Database optimisation (indexing, complex query reduction), reliable hosting, load balancing.
    - **Monitoring/Measuring Performance**: Tools like Google PageSpeed Insights, Lighthouse, WebPageTest.

- **Web Content Management Systems (CMS)**

    - Software that helps users **create, manage, modify website content without technical knowledge**. Central hub for publishing.
    - **How a CMS Functions**:
        - **Content Management Application (CMA)**: Front-end interface for creating/editing content (e.g., WYSIWYG editors).
        - **Content Delivery Application (CDA)**: Back-end component that takes content from CMA, applies templates, formats into final website.
    - **Templates in a CMS**: Control layout, structure, appearance (HTML, CSS, JS).
        - **Themes**: Collections of templates/stylesheets (e.g., WordPress themes).
        - **Template Files**: Correspond to website parts.
        - **Template Tags**: Fetch/display dynamic content from database.
    - **Content Storage**: Typically use **databases** (e.g., MySQL for WordPress).

        - **Database Structure**: Organises content into tables.
        - **Content/Metadata** saved in database; **media files** in folders with database references.
        - **Dynamic Retrieval**: CMS retrieves content dynamically using queries, combines with templates.
    - **WordPress**: Leading **open-source CMS** (over 40% of websites).
        - **Features**: Themes, Plugins (SEO, e-commerce, security), Widgets, Custom Post Types.
    - **Prevalence**: WordPress (62.7%), Shopify (6.4%), Wix (3.9%), Squarespace (3.0%), Joomla (2.4%).
    - **Benefits**: Ease of Use (visual editors), Time Efficiency (pre-built), Scalability (plugins, themes), Community Support.
    - **Limitations**: Resource Intensive (hosting costs), Learning Curve for Customisation (HTML, CSS, PHP knowledge needed).
    - **CMS vs. Traditional Web Development**: CMS offers **code abstraction, lower costs, direct content management**. Traditional offers **maximum control**. CMS can be extended with custom code.
- **Contribution of Back-End Web Development**

    - Creates infrastructure for application's **core functionality** (invisible to users). Processes data, manages databases, executes complex logic.
    - **Key Components**: **Web Servers**, **Databases**, **Application Logic** (handled by frameworks like Django).
    - **Contribution to Success**:
        - **Data Management**: Storing, retrieving, updating, deleting data.
        - **Security**: Authentication, encryption, access control.
        - **Dynamic Content**: Provides data to front end via APIs.
        - **Scalability**: Handles growing users/traffic (load balancing, caching, sharding).
        - **Business Logic**: Processes complex operations, transactions, data validation.
        - **Multi-Platform Support**: Single backend supports various clients (web, mobile, IoT).
        - **Reliability**: Ensures server stability.
    - **Server**: Computer that receives messages from client computers over internet.
    - **Back-end programming languages** needed to program computers as servers.
    - **Back-end frameworks** (Express.js, Django) simplify server creation.
    - **Package Managers** (NPM, PIP) manage libraries.
    - **Request-Response Cycle**: Front-end sends request, back-end processes/saves to database, sends response.
    - **APIs (Application Programming Interfaces)**: "**Glue**" between front-end and back-end. Defines **rules for data requests/responses**.
        - Enable clients to interact with server's logic/data **without direct database access**.

        - **Functions**: Data exchange (JSON/XML), encapsulation/abstraction, integration with third-party services, multi-platform support, standardised communication (REST, GraphQL).
        - **API Endpoints**: Specific URL/URI for a resource/functionality. Uses HTTP methods (GET, POST, PUT, PATCH, DELETE).
    - **ORM (Object-Relational Mapping)**: Allows developers to interact with relational databases using their programming language (e.g., Python objects for Django ORM) **without writing raw SQL**.
        - Converts database rows into objects, tables into classes.
        - Simplifies data manipulation, reduces SQL injection risk, improves code readability.
- **Back-End Process to Manage a Web Request**
    - **1. Client Request**: Browser sends HTTP request.
    - **2. Web Server Role**: Server receives request.
        - For static content, retrieves/sends directly.
        - For dynamic content, forwards to web application framework (e.g., Django) via WSGI.
    - **3. Web Framework Role (Application Logic)**: Framework (e.g., Django) processes dynamic request.
        - Routes request to view/controller.
        - Interacts with objects, libraries, databases.
        - **Django's MVT Architecture Components**:
            - **Models (Data Layer)**: Define database table structure as Python classes; Django's ORM maps to tables.
            - **Views (Business Logic Layer)**: Handle logic, bridge models/templates. Process HTTP requests, interact with models, render templates.
            - **Templates (Presentation Layer)**: Render HTML pages dynamically with data.
            - **Django ORM**: Abstracts database layer, maps tables to Python classes for CRUD.
            - **URLs and Routing**: URL dispatcher maps URLs to views.
            - **Forms**: Handles user input, validation, prevents vulnerabilities.
            - **Middleware**: Functions processing requests/responses (security, session management).
            - **Django Admin**: Built-in interface for managing database data.
            - **Authentication/Authorisation**: Comprehensive user system.
            - **API Integration**: Django REST Framework (DRF) builds/manages APIs.
    - **4. Objects**: Instances of models used to represent/manipulate database records via ORMs.
    - **5. Libraries**: Used in back-end code for specific tasks (data processing, validation, integrations).
    - **6. Databases**: Accessed for data storage/retrieval.
- **Creating Files/Directories & Searching Text with Shell Scripts**

    - **Bash (Bourne Again Shell)**: Suitable scripting language for Unix-based systems (Linux, macOS).
    - **Capabilities**:
        - **File Manipulation/Directory Operations**: `mkdir`, `touch`, `mv`, `cp`, `rm`.
        - **Text Processing Capabilities**: `grep` (search text by patterns), `sed` (substitution), `awk` (processing/reporting).
        - Integration with System Commands.
        - Portability.
    - **Practical Use**: Routinely generate file/folder structures for PWA projects, automate tasks (e.g., starting Node.js server).
- **Web-Based Database and SQL Scripts**
    - **SQL (Structured Query Language)**: Programming language for **managing/manipulating data in relational databases**.
    - **Commands**:
        - **SELECT**: Retrieves data, can specify fields (columns). Example: `SELECT title, artist FROM tracks;`.
        - **GROUP BY**: Groups rows with same values into summaries (e.g., calculate averages).
        - **INSERT INTO**: Adds new records (rows).
        - **UPDATE**: Modifies existing records.
        - **DELETE FROM**: Removes records.
        - **DROP TABLE**: Deletes entire table/database.
    - **Constraints (WHERE keyword)**: Filters records based on conditions.
        - **WHERE**: Filters rows.
        - **AND/OR/NOT**: Combine conditions.
        - **LIKE**: Pattern matching with wildcards (%, _).
        - **IN**: Specifies multiple possible values.
    - **Table Joins**: Combine rows from two or more tables based on related columns.
        - **INNER JOIN**: Returns rows with **matches in both tables**.
        - **LEFT OUTER JOIN**: Returns **all rows from left table**, matching right rows (NULLs if no match).
        - **RIGHT OUTER JOIN**: Returns **all rows from right table**, matching left rows (NULLs if no match).
        - **FULL OUTER JOIN**: Returns **all rows from both tables**, combining matches (NULLs for non-matches).
- **Object-Relational Mapping (ORM) vs. SQL**
    - Both methods for working with databases, distinct purposes.
    - **ORM**: Interacts with relational databases using **programming language objects** (e.g., Python) **without raw SQL**.
    - **Comparison Categories**:
        - **Abstraction**:
            - **ORM**: Abstracts database operations through object models, simplifying interaction.
            - **SQL**: Requires detailed queries, precise control.

        - **Flexibility**:
            - **ORM**: Simplifies database interactions with objects.
            - **SQL**: Offers granular control.
        - **Learning Curve**:
            - **ORM**: Has a learning curve (e.g., Django ORM).
            - **SQL**: Generally straightforward, widely used.
        - **Code Readability/Maintenance**:
            - **ORM**: Integrates database operations within programming language, more readable/consistent.
            - **SQL**: Raw SQL can be complex/hard to maintain when scattered.
        - **Security**:
            - **ORM**: Most ORMs automatically handle input sanitisation, **reducing SQL injection risk**.
            - **SQL**: Requires manual sanitisation to prevent injection attacks.
- **Collaborative Development**

    - **Collaboration between front-end and back-end developers is crucial**.
    - **Work Practices**: Clear communication, regular meetings, feedback loops.
    - **Interconnectivity and Workflow**:
        - **UX to UI**: UX provides wireframes/user flows; UI enhances with visuals.
        - **UI to Front-End**: UI passes prototypes; front-end transforms into interactive code (HTML, CSS, JS).
        - **Version Control (Git)**: Multiple developers work simultaneously using branches, facilitates code reviews.
        - **APIs**: **"Glue"** between front-end and back-end, enabling front-end to request/display data dynamically. Front-end uses HTTP methods to request data. Testing APIs is essential (e.g., Postman).
        - **Cross-functional Collaboration**: Leveraging strengths, sharing knowledge.
        - **Iterative Development**: Build, test, refine features incrementally.
        - **Code Consistency/Integration**: Ensuring common patterns and smooth integration.
- **PWA Design Considerations**

    - PWA combines web platform technologies with platform-specific app experience.
    - **Technical Features**: HTML page, CSS, JavaScript, Web APIs. Optional: web app manifest (installation info), service worker (offline experience).
    - **Design Considerations**: Fast loading, push notifications, offline functionality, responsiveness, engaging UI.
    - **Application of Design/UI/UX Principles**:
        - **User Experience (UX)**: Focuses on **user interaction, functionality, usability, user journey**. UX designers conduct research, create personas, user flows, site structure, wireframes.

        - **User Interface (UI)**: Focuses on **visual design** (layout, visual elements, interaction patterns). UI designers create visual prototypes, style guides, high-fidelity mockups.
        - **Specific Elements Applied**:
            - **Font**: Readable fonts, consistent typography (UI).
            - **Colour**: Color schemes for appeal, brand alignment (UI).
            - **Audio/Video**: Optimisation for network conditions (data compression, encoding).
            - **Navigation**: Intuitive elements (menus, links, buttons).
            - **Accessibility**: Ensuring usability for individuals with disabilities.
                - W3C's **WCAG** provides framework.
                - Considerations: Alt text for images, perceivable/operable/understandable/robust content across assistive technologies (screen readers).

Automation

- **Machine Learning (ML)**
    - **Definition**: A branch of **artificial intelligence (AI)** that develops **algorithms** and **models** enabling computers to **learn from data** and make predictions/decisions without explicit programming.
    - **How it works (keywords)**:
        - **Data**: Foundation; numbers, text, images, audio.
        - **Algorithms**: Mathematical procedures used to learn from data.
        - **Training phase**: Machine learns from a dataset by finding **patterns and relationships**.
        - **Testing phase**: Model evaluated with **new data** to check generalisation performance.
- **Main Types of Machine Learning**
    - **Supervised learning**: Trained on **labelled dataset** (input + correct output). Predicts output based on input. E.g., predicting house prices.
    - **Unsupervised learning**: Trained on data **without labelled outputs**. Finds **patterns or groupings** on its own. E.g., clustering customers.
    - **Reinforcement learning**: Learns by **interacting with an environment**, receives **rewards or penalties** (feedback), updates model to make **optimal decisions**. E.g., game playing, robotics.
- **Applications of Machine Learning**
    - **Image recognition**: Identifying objects, people, scenes (facial recognition).
    - **Natural Language Processing (NLP)**: Understanding human language (chatbots, translation).

    - **Recommendation systems**: Suggesting movies, products based on preferences.
    - **Fraud detection**: Analysing transactions for suspicious activity.
- **Challenges in Using Machine Learning**

    - **Quality of data**: Success heavily depends on quality and quantity of training data.
    - **Overfitting**: Model learns training data too well (including noise), performs poorly on new data.
    - **Bias**: Biased/unrepresentative training data can perpetuate biases in predictions.
- **DevOps vs. MLOps**

    - **DevOps**: Fosters collaboration to create better systems. Focuses on **software development life cycle**.
    - **MLOps**: Focuses on the **ML life cycle**.
    - **Complexity**: ML models are often more complex, requiring specialised tools.
    - **Data**: ML models rely on data for training/inference, introducing data management challenges.
    - **Regulation**: ML models may be subject to regulatory requirements.
- **Artificial Intelligence (AI) vs. Machine Learning (ML)**

    - **AI**:
        - **Broader concept** of machines simulating human intelligence.
        - Goal: Make smart computer systems to solve complex problems like humans.
        - Includes techniques like expert systems, rule-based programming.
        - Can make decisions with limited/no human intervention.
        - Can include systems that don't learn or adapt over time.
        - Applications: Siri, chatbots, intelligent robots.
    - **ML**:
        - **Subset of AI**.
        - Goal: Allow machines to **learn from data** automatically to give accurate output.
        - Always requires **large amounts of data** to function.
        - Primary goal: Enable systems to learn from experience.
        - Focuses on algorithms that **improve through data**.
        - Applications: Recommender systems, Google search algorithms, Facebook auto friend tagging.
- **Variables in Machine Learning**

    - **Variable**: A **characteristic, property, or feature** that changes and represents data.
    - **Features (Independent Input Variables)**: Used by the model to make predictions. E.g., house size, number of bedrooms.
    - **Target Variable (Dependent Output Variable)**: The outcome the model predicts. E.g., house price, Pass/Fail.

- **Traditional Computing vs. Machine Learning**
    - **Traditional Computing**:
        - Purpose: Decision support based on **predefined rules**.
        - Implementation: **Static, rule-based systems**.
        - Adaptability: **Does not adapt** to new information.
    - **Machine Learning**:
        - Purpose: **Predictive modelling** based on **learned patterns**.
        - Implementation: **Dynamic, data-driven learning**.
        - Adaptability: **Learns and improves with new data**.
- **Decision Trees**
    - **Definition**: A **flowchart-like structure** for making decisions based on data, consisting of **nodes** (features), **branches** (decision rules), and **leaves** (outcomes).
    - **How they work**: Algorithm selects features to **split data** into classes, repeating until conditions met. Used for predictions by traversing from root to leaf.
    - **Why software engineers use them**:
        - **Easy to understand and interpret** (visual structure).
        - Works with **numerical and categorical data**.
        - Can identify **important features**.
- **Neural Networks (Artificial Neural Networks - ANNs)**
    - **Definition**: Computational model inspired by biological brains, with **layers of interconnected nodes** (neurons) that process information and learn complex patterns.
    - **Components**:
        - **Node layers**: Input, **hidden**, and output layers.
        - **Nodes (artificial neurons)**: Each is a linear regression model.
        - **Weights**: Numerical values representing **influence** of input on output.
        - **Bias/Threshold**: Part of each node.
        - **Feed forward network**: Data passes from one layer to the next.
    - **How they work**:
        - **Training Cycle**:
            - **Loss function**: Quantifies prediction error.
            - **Backpropagation**: Adjusts network weights to minimise error.
            - **Gradient descent**: Adjusts weights and biases to reduce errors.
        - **Execution Cycle**: Uses trained weights to make predictions on new data via **forward propagation**.
    - **Limitations**:
        - Requires **large amount of data**.
        - **Resource-intensive** training (computational power).
        - Often considered **'black boxes'** (difficult to interpret internal workings).
    - **Types**: Convolutional Neural Networks (CNNs) for image recognition, Recurrent Neural Networks (RNNs) for time series data.

- **Linear Regression**
    - **Purpose**: Predicts a **continuous numerical value** based on continuous input variables.
    - **How it works**: Finds the **best-fitting line** that minimises overall error across all data points.
    - **Use cases**: Predicting house prices, sales forecasting, patient outcomes, student performance, air quality, customer lifetime value.
- **Logistic Regression**
    - **Purpose**: Generates predictions for **categorical data** (e.g., colour, gender).
    - Ideal for **binary classification problems** (e.g., 'cat' or 'not a cat', app usage yes/no, password cracked yes/no).
    - **Data Representation**: Categorical features represented as **binary values (0 or 1)**.
    - **How it works**: Uses the **sigmoid function** to map input features to a **probability between 0 and 1**.
- **K-Nearest Neighbour (KNN)**
    - **Type**: **Supervised learning algorithm**.
    - **Non-parametric**: Makes no assumptions about data distribution.
    - **How it works**: For a new data point, it finds the **'k' closest points (neighbours)** and assigns the new point to the group that most of its neighbours belong to. Primarily used for **classification**.
- **Polynomial Regression**
    - **Purpose**: Used when the relationship between variables is **non-linear**.
    - **Difference from Linear Regression**: Linear assumes a straight line; polynomial **allows for curves** by adding powers of the independent variable.
    - **Degree of the polynomial**: Highest power of 'x' in the equation, determines **flexibility of the curve**. Higher degree can fit data more closely but risks **overfitting**.
    - **How it works**: Input features are **transformed into polynomial features** (e.g., x^2, x^3), then a regression model is fitted.
    - **Use cases**: Car performance (speed vs. fuel), house prices (size vs. price), temperature changes, sales over time, human growth patterns.
- **Bias in Machine Learning**
    - **Human Bias**: Subjective judgements made by individuals during ML/AI development.
        - **Types**:
            - **Selection bias**: Developers choose datasets reflecting their preferences, leading to poor generalisation (e.g., facial recognition on lighter skin).
            - **Confirmation bias**: Prioritising results that confirm existing beliefs.
            - **Cognitive bias**: Subjective judgements influencing feature selection, model design, evaluation.

            - **Implicit bias**: Unconscious biases influencing data (e.g., educators' grading).
        - **Effects**: Inaccurate predictions, perpetuating societal inequalities (e.g., biased hiring algorithms).
    - **Dataset Source Bias**: Data used for training is **unrepresentative** of the target population or real-world scenarios.
        - **Types**:
            - **Sampling bias**: Data collected from specific demographic, not generalisable (e.g., urban health data).
            - **Labelling bias**: Inconsistent or biased labelling criteria by annotators.
            - **Historical bias**: Datasets reflecting past inequalities (e.g., criminal justice data).
            - **Measurement bias**: Inaccuracies/inconsistencies in the data (e.g., grading errors).
            - **Feature bias**: Model relying too heavily on certain features that don't capture full abilities.
        - **Effects**: Disadvantaging specific groups, eroding trust in technology.
        - **Feedback loop**: Predictions influencing future data, reinforcing biases (e.g., student grades).
- **Mitigation Strategies for Bias**

    - Use **explainable AI** techniques.
    - Provide **training** for developers on recognising/mitigating bias.
    - Encourage **diverse teams** in AI development.
    - Conduct **regular audits** and test models on different demographic groups.
    - Implement **fairness metrics**.

SSA

- **Designing Software**
    - **Benefits of developing secure software**
        - Essential for **protecting data**, **minimising vulnerabilities**, **preventing cyber attacks**.
        - Safeguards sensitive information from **unauthorised access, manipulation, theft**.
        - **Data Protection**
            - **Encryption**: Data at rest/in transit.
            - **Access Controls**: Role-based access control (RBAC).
        - **Confidentiality**: Data private, protected from unauthorised access.
        - **Integrity**: Prevents unauthorised data modification/tampering.
        - **Availability**: Data/services accessible to legitimate users.
        - **Compliance with regulations**: GDPR, HIPAA, PCI DSS.
        - **Customer trust**: Enhances loyalty and satisfaction.
        - **Business continuity**: Reduces data loss/service disruptions.
        - **Minimising cyber attacks & vulnerabilities**: Secure coding, patching.
        - **Prevent data breaches**: Protects sensitive info.
        - **Mitigate against cyber threats**: Reduces malware, ransomware, phishing, DoS.
        - **Cost savings**: Addressing security early is more cost-effective.
        - **Competitive advantage**: Differentiates organisations.
        - **Vulnerabilities**: Weaknesses exploited (e.g., SQL Injection, XSS). Avoided by **input validation, parameterised queries, output encoding**.
- **Fundamental software development steps (SDLC)**
    - Security must be **embedded from the beginning**.
    - Phases: **Requirements, Specifications, Design, Development, Integration, Testing & Debugging, Installation, Maintenance**.
    - **Requirements definition**: Focus on **non-functional security requirements** (e.g., password hashing).
    - **Determining specifications**: Translate requirements into actionable details (e.g., **strong password hashing, rate limiting, HTTPS**).
    - **Design**: Define architecture & components.
        - **Key design principles**: **Modularity, Abstraction, Encapsulation, Reusability, Maintainability**, **MVC Pattern** (Model, View, Controller).
    - **Development**: Code written per design.
    - **Integration**: Modules tested together; security assessment of existing systems, secure APIs/databases.
    - **Testing and Debugging**: Identify/fix vulnerabilities (e.g., SQL injection).
    - **Installation**: Secure process. Key practices: **Secure Connections (HTTPS), Environment Isolation, Database Configuration, Access Control/Permissions**.

    - **Maintenance**: Regular updates, security patches, continuous monitoring.
- **How end-user capabilities/experience influence secure design features**
    - **User-centered design (UCD)**: Puts end-user first, minimises security risks from misuse.
    - **UCD principles for secure software**: **Simplicity, Clarity, Transparency**.
    - **Common user behaviours affecting security & solutions**:
        - **Weak password practices**: Implement strong policies.
        - **Phishing/social engineering**: User education.
        - **Failure to update software**: Encourage automatic updates.
        - **Bypassing security protocols**: Design user-friendly protocols.
    - **Influence on design features**: **User-friendly authentication** (e.g., MFA options), **Role-Based Access Control**, **Training and awareness**, **User feedback/testing**, **Simplified security controls**.
- **Fundamental software design security concepts**
    - **CIA Triad**:
        - **Confidentiality**: Data private. Ensure via **encryption, HTTPS**.
        - **Integrity**: Data accurate, consistent, trustworthy. Ensure via **checksums, digital signatures, hash functions**.
        - **Availability**: Services accessible. Ensure via **redundancy, load balancing, disaster recovery**.
    - **AAA**:
        - **Authentication**: Verifies user identity. Ensure via **strong mechanisms (passwords, 2FA, biometrics)**.
        - **Authorisation**: Enforces access control based on roles. Ensure via **RBAC/ABAC**.
        - **Accountability**: Traces actions to individual. Ensure via **logging, auditing**.
- **Security features incorporated into software**
    - **Data protection**: **Secure password hashing (Bcrypt), data encryption (TLS)**.
    - **Security measures**: **Input validation/sanitisation** (prevent SQLi/XSS), **parameterised queries**, secure coding practices.
    - **Privacy protection**: Clear privacy policy, user consent, data anonymisation.
    - **Regulatory compliance**: Align with laws (e.g., Privacy Act 1988), regular audits.
    - **User authentication & authorisation**: **MFA**, access controls, monitoring user access.
- **Contribution of Cryptography & Sandboxing to 'Security by Design' (SbD)**
    - **Security by Design (SbD)**: Integrates security controls **from design phase**, not an afterthought.
    - **Cryptography**: Encodes info. Ensures **confidentiality, integrity, authenticity**. Used for **data encryption, password hashing**.
    - **Sandboxing**: Isolated environment for safe code testing. Limits breach impact. E.g., web browser sandboxing, isolating third-party APIs.

- **‘Privacy by Design’ (PbD) approach**
    - Proactive approach, integrates privacy from outset.
    - **Proactive not reactive**: Anticipates issues during design.
    - **Embed privacy into design**: Protections built-in by default (data minimisation, anonymising).
    - **Respect for user privacy**: User interests paramount, user control over data.
- **Testing and evaluating software security and resilience**
    - **Vulnerabilities**: Weaknesses attackers exploit.
    - **Resilience**: Ability to withstand and recover from failures/attacks.
    - **Determining vulnerabilities**: Identify weaknesses via tools, inspection, simulated attacks.
    - **Hardening systems**: Adopting best practices.
    - **Handling breaches**: **Incident response plan** (detect, contain, mitigate).
    - **Maintaining business continuity**: Critical operations during/after disruption.
    - **Conducting disaster recovery (DR)**: Restoring systems after major failure/attack. Involves RTO, RPO.
- **Strategies used by software developers to manage security**
    - **Code review**: Systematic examination to identify bugs, improve quality, ensure secure coding.
    - **Static Application Security Testing (SAST)**: Analyses source code *without executing* to find vulnerabilities early (e.g., SQLi, unvalidated input). **Advantages**: Early detection, immediate feedback.
    - **Dynamic Application Security Testing (DAST)**: Tests running application by simulating attacks (e.g., XSS, SQLi, weak session management). **Advantages**: Finds runtime vulnerabilities, realistic assessment.
    - **Vulnerability assessment**: Scans to identify, quantify, prioritise weaknesses.
    - **Penetration testing (Pen testing)**: Proactive simulation of real-world attacks to exploit vulnerabilities. Assesses system resilience.
- **Defensive data input handling practices**
    - **Input validation**: Verifies user input format/constraints.
        - Types: Client-side, **Server-side (critical for security)**.
        - Best Practice: **Never trust user input, use whitelist validation**.
    - **Sanitisation**: Cleans/escapes harmful characters/scripts (e.g., for XSS, SQLi).
    - **Error handling**: Manages errors gracefully.
        - Key Principles: **Do not expose sensitive information**, **graceful error recovery**, **log errors securely**.
- **Safe Application Programming Interface (API) design & implementation**
    - **Common API vulnerabilities**: Lack of Auth/Auth, Data Exposure, Insecure Communication, Injection Attacks, Lack of Rate Limiting.
    - **Best Practices for Securing APIs**: **Authentication/Authorisation (OAuth 2.0), HTTPS, Input Validation/Sanitisation, Limit Data Exposure, Rate Limiting, API Keys/Tokens, Audit/Monitor API Usage**.

- **Code considering efficient execution for the user**
    - **Memory management**: Wise resource use, avoids leaks, protects sensitive data in memory.
    - **Session management**: Tracks user interactions, authenticates.
        - Concepts: **Session IDs** (unique, random), **Session Timeout**, **Session Regeneration**, **Secure Cookies** (HttpOnly, Secure).
        - Vulnerabilities: **Session Hijacking, Session Fixation**.
    - **Exception management**: Handles unexpected problems gracefully, prevents crashes, protects from threats.
- **Secure code to minimise vulnerabilities in user action controls**
    - **Broken authentication & session management**: Allows unauthorised access.
        - Prevention: **Strong password policies, MFA, secure session management**.
    - **Cross-Site Scripting (XSS)**: Injecting malicious scripts into web pages.
        - Prevention: **Sanitise input, Output encoding**.
    - **Cross-Site Request Forgery (CSRF)**: Attacker tricks user into unintended actions.
        - Prevention: **CSRF Tokens, SameSite Cookies**.
    - **Invalid forwarding and redirecting**: Information not found in source.
    - **Race conditions**: Concurrent access to shared resources leads to unpredictable outcomes.
        - Prevention: Use **locks**.
- **Secure code to protect user file & hardware vulnerabilities (File Attacks & Side Channel Attacks)**
    - **File attacks**: Improper handling of uploads, permissions.
        - Vulnerabilities: Malicious File Uploads, Path Traversal, Improper File Permissions, File Name Injection.
        - Prevention: **File Type Validation, File Name Sanitisation, Limit File Permissions, Random File Names, Secure Directories, Access Control, Malware Scanning, Size Limits, Encryption**.
    - **Side channel attacks**: Gather info from physical implementation, not direct software flaws (e.g., timing, power, cache).
        - Mitigation: **Constant-Time Algorithms, Blinding Techniques, Random Delays, Shielding, Proper Cache Management, Hardware Isolation (TEE), Firmware Updates, Secure Boot, Physical Security**.
- **Impact of safe and secure software development**
    - **Benefits of collaboration**: Diverse points of view, task delegation by expertise, improved solution quality.
    - **Benefits to an enterprise**: Improved products/services, influence on future development, improved work practices, productivity, business interactivity (enables remote work).
    - **Social, ethical, and legal issues**:

- **Employment**: Job creation (cybersecurity) vs. displacement (AI/automation).
- **Data security**: Safeguarding personal data (identity theft prevention).
- **Privacy**: Protecting personal info (consent, GDPR).
- **Copyright**: Legal rights to original works (software code).
- **Intellectual property (IP)**: Protecting creations (patents, trademarks, licensing).
- **Digital disruption**: Impact on business models, potential digital divide.

Program for Web

Here are very concise, indented dot-point notes on the provided source:

- **Programming for the Web**
    - **Applications of Web Programming**
        - **Interactive Website/Webpages**: Enables **dynamic functionality** for browsing, communication, and interaction online.
            - Examples: Social media, online learning, web-based games, project management, government/public service.
            - Focus: User engagement, data management, productivity.
        - **E-commerce**: Facilitates **buying and selling goods/services** over the internet.
            - Examples: Amazon, eBay, Alibaba.
            - Key components: Internet access, mobile tech, secure payment gateways, UX innovations (recommendations, reviews).
        - **Progressive Web Apps (PWAs)**: Combines **web and mobile app features**.
            - **Installable** on devices without app stores.
            - **Faster, more reliable**, and offer **offline functionality**.
            - Built with **HTML, CSS, JavaScript, Web APIs**.
            - **Adapt to screen sizes** and **stay up-to-date**.
            - Can offer **push notifications** and **full-screen mode**.
            - **Recognition**: Installation prompt, home screen icon, full-screen, push notifications, offline, responsive design.
            - Built with **progressive enhancements**.
- **Data Transfer on the Internet**
    - **Data Packets**: Data (e.g., email, video) is broken into **smaller units** for efficient, reliable, scalable transmission.
        - Each packet contains **part of data + additional info** (e.g., for reassembly).
        - **Components**:
            - **Header**: **Metadata** (Source/Destination IP, Sequence Number, Protocol Info, Error Checking).
            - **Payload**: The **actual data**.
            - **Trailer**: Indicates **end of packet** (optional error-checking).
        - **Packet Switching**: Packets **take multiple routes**; routers direct them, enabling **dynamic routing**.
            - Fundamental for efficient, reliable internet communication.
            - **Key Features**: Data division (typically 1,500 bytes), independent routing, reassembly (sequence numbers), error detection/retransmission, shared network resources.
            - **Advantages**: Efficiency, reliability, error recovery, scalability.

        - **Disadvantages**: Variable delays, potential for packet loss.
        - **Applications**: Internet, Cellular Networks (4G/5G), Email Systems.
    - **Internet Protocol (IP) Addresses**
        - **IP**: Principal **communication protocol for relaying data packets** across networks.
        - **IP address**: Unique **postal address for a device** on the internet.
        - **IPv4**: **32-bit address**, ~4.29 billion unique addresses (e.g., 192.168.1.1). Running out.
        - **IPv6**: **128-bit address**, 340 undecillion addresses (e.g., 2001:0db8:...).
        - **Key Differences (IPv4 vs. IPv6)**: Format (decimal vs. hexadecimal), address count (limited vs. unlimited), IPv6 has built-in security, better mobile support.
    - **Domain Name System (DNS)**
        - **DNS**: Translates **user-friendly domain names** (e.g., "www.example.com") into **IP addresses** for computers. Acts as internet's "address book".
        - **How it Works**:
            - User enters domain.
            - Browser contacts **DNS resolver** (often ISP).
            - Resolver checks **DNS cache**.
            - If not cached, resolver queries **hierarchy of DNS servers** (root, TLD, authoritative) to get IP.
            - IP returned to browser, enabling connection.
        - **Common DNS Records**:
            - **A record**: Maps domain to **IPv4**.
            - **AAAA record**: Maps domain to **IPv6**.
            - **CNAME record**: Maps domain to **another domain**.
            - **MX record**: Specifies **mail servers**.
        - **Port**: DNS typically uses **Port 53**.
- **Web Protocols and Their Ports**
    - **Importance**: Facilitate internet communication, operate on designated ports, crucial for software engineers to design efficient, reliable, secure applications.
    - **HTTP (Hypertext Transfer Protocol)**: Allows browsers to **fetch resources** (e.g., HTML pages) from web servers.
        - **Stateless Protocol**: Each request is independent; server doesn't remember previous requests.
        - **Unsecured**: Data can be intercepted.
        - **Port**: Typically uses **Port 80**.
    - **HTTPS (Hypertext Transfer Protocol Secure)**: **Secure version of HTTP**.
        - Uses **SSL/TLS encryption** for secure data transmission.
        - **Crucial for sensitive info** (login, payment).
        - **Security Indicator**: Padlock icon in browser.
        - **Port**: Commonly uses **Port 443**.
    - **TCP/IP (Transmission Control Protocol/Internet Protocol)**: **Foundational communication protocol suite** for the internet.

        - **TCP**: Responsible for **reliable data transmission**; breaks data into packets, reassembles them, establishes connection (connection-oriented). Example: file downloads.
        - **IP**: Handles **addressing and routing** data packets to destination.
        - **Difference**: IP defines address, TCP transports data reliably to that address.
        - **Transport Protocol**: Web servers primarily use **TCP** for reliable, ordered, error-checked delivery.
    - **FTP (File Transfer Protocol)**: Used for **transferring files** between client and server (uploading, downloading, managing).
        - **Ports**: Uses **Ports 20** (data) and **21** (control).
    - **SFTP (Secure File Transfer Protocol)**: **Secure version of FTP**, encrypted over SSH.
        - **Port**: Uses **Port 22** (same as SSH).
        - **Key Point**: Preferred over FTP for sensitive data due to confidentiality and integrity.
    - **SSL (Secure Sockets Layer) / TLS (Transport Layer Security)**: **Encryption protocols** for internet communications.
        - **TLS is successor to SSL**.
        - Uses **cryptographic algorithms** to scramble data.
        - **Authenticates server's identity** (prevents man-in-the-middle attacks).
        - **No specific ports**, but used with protocols like HTTPS (443), SMTP (25, 587), IMAP (993).
    - **SMTP (Simple Mail Transfer Protocol)**: Used for **sending email messages** between servers.
        - **Port**: Default is **Port 25**.
    - **POP3 (Post Office Protocol version 3)**: Email retrieval protocol; **downloads emails** to device, often removing from server.
        - **Port**: Uses **Port 110**.
    - **IMAP (Internet Message Access Protocol)**: Email retrieval protocol; allows **viewing emails remotely** on server, synchronising across devices.
        - **Port**: Uses **Port 143**.
    - **Secure Versions (email)**: POP3S (995), IMAPS (993) use SSL/TLS.
- **Securing the Web**

    - Ensures **confidentiality, integrity, availability** of data/systems. Often legally required (e.g., GDPR).
    - **SSL Certificates (now TLS)**: **Digitally bind a cryptographic key** to an organisation's details, enabling **secure HTTPS connections**.
        - **Digital certificate** that authenticates website's identity.
        - Issued by **Certificate Authority (CA)**.
        - Contains domain, organisation, CA, public key, expiration.
        - **How they work**: Browser requests secure connection, server sends certificate, browser verifies validity, handshake generates **session key**, then **encrypted communication**.
        - Ensure secure browsing (encryption), authentication, data integrity.

- **Encryption Algorithms**: Convert **readable data (plaintext) into unreadable (ciphertext)**.
  - **Symmetric Encryption**: Uses **same key** for encryption/decryption. Example: **AES**.
  - **Asymmetric Encryption**: Uses **two keys (public for encryption, private for decryption)**. Example: **RSA**. Used in SSL/TLS for key exchange.
- **Encryption Keys**: Sequences of characters determining algorithm output; must be kept secure.
  - **Public Key**: Shared openly to encrypt.
  - **Private Key**: Kept secret to decrypt.
  - Longer keys (e.g., 128-bit, 256-bit) provide stronger security.
  - **Key Exchange**: Securely sharing keys; SSL/TLS uses asymmetric for exchange, symmetric for data transfer. Diffie-Hellman algorithm.
- **Plaintext and Ciphertext**:
  - **Plaintext**: **Original, readable** data.
  - **Ciphertext**: **Unreadable format** after encryption.
- **Authentication and Authorisation**:
  - **Authentication**: **Verifies who a user is** (e.g., username/password, biometrics, 2FA).
  - **Authorisation**: **Determines what an authenticated user can access** (permissions).
  - Together, prevent unauthorised access.
  - **OAuth**: Authorisation framework for apps to access resources without sharing passwords, uses access tokens.
  - **OpenID Connect (OIDC)**: Builds on OAuth for authentication, used for Single Sign-On (SSO).
- **Hash Values**: Fixed-size string output from input data via a **hash function**.
  - **Properties**: Deterministic, Irreversible, Collision-Resistant.
  - **Uses**: Verifying data integrity (file checksums), securely storing passwords (store hashed passwords).
  - Algorithms: SHA-256, MD5.
- **Digital Signatures**: Verify **authenticity and integrity** of message/document.
  - Created using sender's **private key**, verified with sender's **public key**.
  - Provide: Authentication, Integrity, Non-Repudiation.

- **Effect of Big Data on Web Architecture**

  - **Big Data**: **Massive datasets** (Volume, Variety, Velocity, Veracity) that traditional methods can't manage.
  - **Four Vs**:
    - **Volume**: Amount of data (terabytes, petabytes). Challenges: Storage, processing.
    - **Variety**: Types/formats (structured, semi-structured, unstructured). Challenges: Integration, analysis.
    - **Velocity**: Speed of generation/processing (real-time). Demands real-time processing systems.
    - **Veracity**: Quality, trustworthiness, reliability. Challenges: Ensuring accuracy (filtering, cleaning).

    - **Opportunities**: Customer personalisation, predictive analytics, enhanced decision making.
    - **Challenges**: Storage/processing (distributed computing, cloud), data integration/analysis (ML, data mining), real-time processing.
    - **Effect on Web Architecture**: Driven evolution to more **scalable, flexible, real-time** systems.
    - **Data Mining**: Discovering **patterns, trends, insights** from large datasets.
        - **Applications**: Describing target data, predicting outcomes, fraud detection, user learning, bottleneck detection.
        - **Techniques in Streaming**:
            - **Association**: Recommends items consumed together (e.g., Spotify song sequences).
            - **Classification**: Categorises users (e.g., "hip-hop fans").
            - **Clustering**: Groups users by shared habits without predefined categories (e.g., mood playlists).
            - **Predictive Analytics**: Anticipates user behavior (e.g., Discover Weekly).
        - **Benefits (Streaming)**: Enhanced personalisation, optimised content delivery, business intelligence.
        - **Challenges (Streaming)**: Data privacy, scalability, real-time analysis.
    - **Metadata**: "**Data about data**".
        - **Purposes**: Descriptive (title, author), Structural (organisation), Administrative (access rights).
        - **Relationship with Big Data**: Manages overwhelming info; helps organise/retrieve user activity, preferences efficiently.
        - **Effect on Web Apps**: Enables performance/scalability, data processing/personalisation, search/filtering, security/privacy concerns.
        - **Role in Web Architecture**: Data organisation, enhanced searchability, data governance/compliance.
    - **Streaming Service Management**: Real-time data processing for immediate responses (e.g., financial transactions, social media feeds).
        - **How Streaming Services Manage Big Data**:
            - **Data Storage/Retrieval**: Distributed server networks (CDNs) for low latency.
            - **Real-Time Data Processing**: Caching, buffering, CDNs for seamless experience.
            - **Data Compression/Encoding**: Reduce file sizes (codecs: H.264/H.265, AAC). Adaptive bitrate streaming.
            - **Personalisation/Recommendations**: ML algorithms, data mining (collaborative, content-based filtering).
- **Designing Web Applications**

    - **World Wide Web Consortium (W3C)**: International organisation developing **open standards** for web's long-term growth.
        - **Standards** cover HTML, CSS, XML, SVG, Semantic Web, ensuring accessibility, interoperability, security.

        - **Web Accessibility Initiative (WAI)**: Makes web accessible to individuals with disabilities. **Web Content Accessibility Guidelines (WCAG)**.
        - **Internationalisation (I18n)**: Standards for multilingual content, diverse cultural conventions in PWAs.
        - **Web Security**: Standards (HTTPS, CSP, CORS, secure authentication) for secure PWAs, mitigating XSS, CSRF, injection attacks.
        - **Privacy**: Guidelines for handling sensitive PWA data (user preferences, location), user consent.
        - **Machine-Readable Data**: Standards (XML, SVG) for representing/exchanging data, enhancing interoperability, search engine visibility.
- **Web Development System Elements**
    - Web app structure: **Front-end (client-side) and Back-end (server-side)**.
    - **Interaction**: **Client Request → Server Processing → Server Response → Browser Renders**.
        - **Client Request**: User initiates (URL, link), browser sends HTTP request (resource, headers). Can be GET (retrieve) or POST (send data).
        - **Server Processing**: Web server receives, processes request. For dynamic pages, runs server-side code (Python, PHP, Node.js), interacts with database.
        - **Server Response**: Server sends HTTP response (status code, data e.g., HTML/JSON, headers).
        - **Browser Renders**: Parses HTML (DOM), loads resources (CSS, JS, images), executes client-side scripts.
    - **Code**:
        - **Static Code**: Hard-coded HTML, **unchanging**.
        - **Dynamic Code**: Generated from database/CMS by server-side languages, **changes based on conditions**.
    - **Delivery**:
        - **Static Delivery**: Page pre-generated, stored on server/CDN, **sent directly to browser (fast)**.
        - **Dynamic Delivery**: Server generates page **on-the-fly** in response to requests (for external/user-specific data).
    - **Client Browser Interaction**:
        - **Static Client Browser**: Content same for all users, no interaction beyond viewing.
        - **Dynamic Client Browser**: Uses **JavaScript** for real-time changes based on user interactions or asynchronous data fetching (AJAX).
    - **Client-Side (Front-End) Web Programming**: Code runs on **user's device** (web browser). Creates **interactive parts** users see.
        - **Languages/Technologies**:
            - **HTML**: Structures information.
            - **CSS**: Styles webpage (layout, colours, fonts).

            - **JavaScript**: Primary language for client-side, makes pages interactive. Manipulates DOM, fetches data asynchronously (AJAX), validates input.
        - **Less secure** (visible to users), **fast for small tasks** (no server communication).
    - **Server-Side (Back-End) Web Programming**: Handles operations on a **web server**.
        - Processes requests requiring database interaction, manages databases, security, file operations.
        - **Languages/Technologies**: Python (Django, Flask), Node.js, PHP, Ruby (Rails), Java (Spring Boot).
        - **More secure** (hidden from users), generally **slower** (server-client communication).
    - **Interfacing with Databases (SQL/NoSQL)**
        - **Database**: Structured collection of data, easily accessed/managed. Web server interacts via language like SQL.
        - **Relational Databases (SQL)**: Tabular format (tables, rows, columns).
            - Data points have **unique IDs (keys)**. Relationships via primary/foreign keys.
            - Ideal for **structured data**. Use **SQL queries**.
            - Follow **ACID principles** (Atomicity, Consistency, Isolation, Durability) for reliability.
            - Require **schema defined upfront**.
            - Examples: MySQL, PostgreSQL, SQLite, MS SQL Server, Oracle, MariaDB, Amazon Aurora.
            - **Entities**: Real-world objects (e.g., Car, Course) modeled as classes/tables.
            - **Attributes**: Properties (fields) of an entity.
            - **Instances**: Actual data records (rows).
            - **Entity Relationships**: One-to-One, One-to-Many (ForeignKey), Many-to-Many (ManyToManyField).
        - **NoSQL Databases**: **Non-relational**, non-tabular format.
            - **Flexible schema**, **scalability**, high performance for **unstructured/semi-structured data**.
            - **Schema-Free**, **Horizontal Scalability**.
            - **Diverse Data Models**: Document-Oriented (MongoDB), Key-Value (Redis), Column-Family (Cassandra), Graph (Neo4j).
            - Used for real-time analytics, content management, big data.
            - Example: MongoDB (flexible JSON-like documents).
        - **SQLite**: Lightweight, self-contained, serverless, relational database. Stores database in single file. Default in Django.
        - **Managing Data**: Django admin interface or SQLite shell (SQL commands/Django ORM).
- **Influence of Web Browser on Web Development**
    - Browsers enable **visual interaction** for non-technical users.
    - **Influence**:

        - **Rendering Engine Compatibility**: Developers ensure consistent rendering across engines (Blink, Gecko, WebKit).
        - **CSS/HTML Standards**: Browsers adhere to W3C standards; developers write compliant code.
        - **JavaScript Execution**: JavaScript engine execution; developers write efficient, cross-browser JS.
        - **Feature Detection/Progressive Enhancement**: Leverage modern capabilities, provide fallbacks for unsupported features.
        - **Cross-browser Testing**: Essential for ensuring apps work across different browsers/versions.
        - **Debugging/Optimisation**: Browsers provide tools.
    - **Developer (Dev) Tools**: Built-in browser tools for debugging, testing, optimising.
        - **Features**: Elements Inspectors (HTML), Style Editors (CSS), JavaScript Consoles (JS debugging), **Network Monitors (Network Tab)**.
        - **Network Monitor**: Analyses network requests/responses, identifies slow resources, observes caching, file sizes. Key metrics: Load Time, File Size/Type, HTTP Status Codes.
        - Performance Profilers, tools for responsiveness testing, device/network simulation.
- **Cascading Style Sheets (CSS)**
    - Controls **appearance and style** of webpage, **separates content (HTML) from design**.
    - Controls colour schemes, typography, spacing, layout (UI). Supports **responsive designs** (Flexbox, CSS Grid). Creates transitions/animations.
    - **Consistency of Appearance**: Ensures **uniform styling** across application. CSS variables, frameworks (Tailwind CSS) aid consistency.
    - **Flexibility with Browsers/Display Devices**: Enables **responsive design** (display well on any screen size).
        - **Key Techniques**:
            - **Media Queries**: Apply styles based on **screen size**.
            - **Fluid Layouts**: Use percentages/vw/vh units for widths/heights.
            - **Mobile-First Approach**: Design for mobile first.
        - **Layout Models**:
            - **Flexbox**: **One-dimensional** system for aligning items in row/column. Ideal for nav bars, buttons.
            - **CSS Grid**: Powerful, **two-dimensional** system for rows/columns simultaneously. Ideal for complex layouts (galleries, dashboards).
        - **Positioning**: Precise control over element placement (static, relative, absolute, fixed).
    - **CSS Maintenance Tools**: Simplify and organise CSS, streamline styling.
        - **SASS (Syntactically Awesome Style Sheets)**: CSS preprocessor, extends CSS capabilities.

        - **Benefits**: Variables (reusable values), Nesting (logical grouping), Partials/Import (split files), Mixins (reusable styles), Inheritance (@extend).
        - **Compiling**: Needs a SASS compiler (Node-sass, Dart-sass) to convert to standard CSS.
    - **Tailwind CSS**: **Utility-first framework**. Provides **utility classes for rapid styling directly in HTML** (e.g., bg-blue-500).
        - **Benefits**: Efficiency, Consistency, Customisation.
        - **Contrast with SASS**: SASS extends CSS; Tailwind shifts styling to HTML with pre-defined classes.
- **Version Control**
    - Systems (like Git) **manage changes** to code, allowing multiple developers to work simultaneously without overwriting.
    - **Reasons for Importance**:
        - **Collaboration**: Concurrent work, merging changes, conflict resolution.
        - **History Tracking**: Review previous versions, compare changes, revert to earlier states (debugging, auditing).
        - **Backup/Disaster Recovery**: Protects against data loss.
        - **Branching/Experimentation**: Separate development lines for features/bug fixes.
        - **Code Reviews/Quality Assurance**: Platform for sharing code and peer reviews.
        - **Deployment/Continuous Integration**: Integrates with automated processes.
        - **Traceability/Compliance**: Audit trail of code changes.
- **Code Libraries for Front-End Web Development**
    - **Libraries**: Collections of **functions/methods for specific purposes** that developers selectively use.
    - **Frameworks**: Provide a **structure/blueprint** for developing applications, enforce coding patterns. Front-end frameworks streamline UI development.
    - **Significance**: Efficiency (reuse code), Consistency (predefined conventions), Scalability (structured solutions), Maintainability (organisation), Accessibility.
    - **Frameworks for Complex Web Applications**:
        - **React**: JavaScript library (often considered framework) for **dynamic, responsive user interfaces**. Emphasises **component-based architecture** (reusable UI modules).
            - **JSX (JavaScript XML)**: HTML-like code in JS files.
            - **Props**: Immutable data passed parent-to-child.
            - **State**: Mutable data within a component; changes trigger re-rendering.
            - **Services**: Handle business logic, data fetching.
            - **Dependency Injection (DI)**: Supplies components/services with needed objects.
            - **Two-Way Data Binding**: Seamless communication between view and model.

            - **Strict Structure**: Enforces structure for scalability/maintainability.
            - **Django and React Together**: Django (backend) + React (frontend).
                - **Django**: Server-side operations, database, API development (REST Framework), authentication, business logic.
                - **React**: Client-side rendering, user interactions, displaying data from Django APIs, state management. Fetches data using HTTP (Axios, Fetch API).
                - **Separate Servers**: React dev server (e.g., localhost:3000) for hot-reloading.
                - **Integration in Production**: Build React app and serve static files with Django, or deploy separately.
                - **Benefits**: Dynamic UI, reusability, modern practices, flexibility, "best of both worlds".
    - **Template Engines**: Streamline **HTML generation**, combine HTML templates with data to produce **dynamic HTML**. Separates HTML layout from business logic.
        - **Django Template Language (DTL)**: Django's built-in engine.
        - **Inheritance**: `{% block %}` and `{% extends %}` for base/child templates (consistency, reusability).
        - **Benefits**: Dynamic content, separation of logic/presentation, security (auto-escapes XSS), reusable components.
        - **Context Processors**: Provide extra context data globally across templates (e.g., user info).
        - Other engines: Handlebars, Mustache, Pug (client-side).
    - **Predesigned CSS Classes**: Provided by **CSS frameworks** (Bootstrap, Foundation, Bulma).
        - Offer predesigned components/layouts for rapid prototyping.
        - Example: **Tailwind CSS** (utility-first framework) provides specific, reusable classes for direct HTML styling.
- **Open-Source Software (OSS)**
    - Source code is **open for inspection, modification, enhancement**. Hosted on GitHub, GitLab, Bitbucket.
    - **Benefits**: Accessibility, community-driven innovation, cost-effectiveness.
    - **Impact on Web Development**: Instrumental due to accessibility, innovation, cost-effectiveness. Many popular web technologies are open source (Bootstrap, React, Angular, jQuery).
    - **Famous Projects**: Linux, Apache HTTP Server, WordPress, React, Angular, VS Code.
- **Support and Manage Load Times**
    - **Web Performance Metrics (Core Web Vitals)**:
        - **Largest Contentful Paint (LCP)**: Measures **main content loading performance**. Good: ≤ 2.5s.

        - Improvements: Optimise images/videos (compress, WebP, lazy-load), CDN, minimise render-blocking resources, Server-Side Rendering (SSR).
    - **First Input Delay (FID)**: Time for page to **respond to first user interaction**. Good: ≤ 100ms.
        - Improvements: Reduce JavaScript execution (minify, defer), split long tasks, optimise third-party scripts, Web Workers.
    - **Cumulative Layout Shift (CLS)**: Measures **unexpected visual stability** (elements shifting position). Good: ≤ 0.1.
        - Improvements: Specify dimensions for images/videos, reserve space for dynamic content.
- **Methods to Support/Manage Load Times**:
    - **Compression**: Gzip, Brotli compress files server-side.
    - **Code Splitting**: Breaks JavaScript into smaller parts, loads when needed.
    - **Lazy Loading**: Loads resources (images, videos) only when in view. `loading="lazy"` attribute.
    - **Asynchronous/Deferred Loading of JavaScript**: Prevents JS from blocking HTML rendering.
        - `async`: Downloads in parallel, executes immediately.
        - `defer`: Downloads in parallel, executes **after HTML parsing complete**.
    - **CDN Caching**: Stores content on edge servers worldwide, reduces load, improves speed. **Content Delivery Network (CDN)** distributes content globally. Example: Cloudflare.
    - **Image Optimisation**: Compress, use efficient formats (WebP), Lazy Loading.
    - **Reducing Server Response Time**: Database optimisation (indexing, complex query reduction), reliable hosting, load balancing.
    - **Monitoring/Measuring Performance**: Tools like Google PageSpeed Insights, Lighthouse, WebPageTest.

- **Web Content Management Systems (CMS)**

    - Software that helps users **create, manage, modify website content without technical knowledge**. Central hub for publishing.
    - **How a CMS Functions**:
        - **Content Management Application (CMA)**: Front-end interface for creating/editing content (e.g., WYSIWYG editors).
        - **Content Delivery Application (CDA)**: Back-end component that takes content from CMA, applies templates, formats into final website.
    - **Templates in a CMS**: Control layout, structure, appearance (HTML, CSS, JS).
        - **Themes**: Collections of templates/stylesheets (e.g., WordPress themes).
        - **Template Files**: Correspond to website parts.
        - **Template Tags**: Fetch/display dynamic content from database.
    - **Content Storage**: Typically use **databases** (e.g., MySQL for WordPress).

        - **Database Structure**: Organises content into tables.
        - **Content/Metadata** saved in database; **media files** in folders with database references.
        - **Dynamic Retrieval**: CMS retrieves content dynamically using queries, combines with templates.
    - **WordPress**: Leading **open-source CMS** (over 40% of websites).
        - **Features**: Themes, Plugins (SEO, e-commerce, security), Widgets, Custom Post Types.
    - **Prevalence**: WordPress (62.7%), Shopify (6.4%), Wix (3.9%), Squarespace (3.0%), Joomla (2.4%).
    - **Benefits**: Ease of Use (visual editors), Time Efficiency (pre-built), Scalability (plugins, themes), Community Support.
    - **Limitations**: Resource Intensive (hosting costs), Learning Curve for Customisation (HTML, CSS, PHP knowledge needed).
    - **CMS vs. Traditional Web Development**: CMS offers **code abstraction, lower costs, direct content management**. Traditional offers **maximum control**. CMS can be extended with custom code.
- **Contribution of Back-End Web Development**

    - Creates infrastructure for application's **core functionality** (invisible to users). Processes data, manages databases, executes complex logic.
    - **Key Components**: **Web Servers**, **Databases**, **Application Logic** (handled by frameworks like Django).
    - **Contribution to Success**:
        - **Data Management**: Storing, retrieving, updating, deleting data.
        - **Security**: Authentication, encryption, access control.
        - **Dynamic Content**: Provides data to front end via APIs.
        - **Scalability**: Handles growing users/traffic (load balancing, caching, sharding).
        - **Business Logic**: Processes complex operations, transactions, data validation.
        - **Multi-Platform Support**: Single backend supports various clients (web, mobile, IoT).
        - **Reliability**: Ensures server stability.
    - **Server**: Computer that receives messages from client computers over internet.
    - **Back-end programming languages** needed to program computers as servers.
    - **Back-end frameworks** (Express.js, Django) simplify server creation.
    - **Package Managers** (NPM, PIP) manage libraries.
    - **Request-Response Cycle**: Front-end sends request, back-end processes/saves to database, sends response.
    - **APIs (Application Programming Interfaces)**: "**Glue**" between front-end and back-end. Defines **rules for data requests/responses**.
        - Enable clients to interact with server's logic/data **without direct database access**.

        - **Functions**: Data exchange (JSON/XML), encapsulation/abstraction, integration with third-party services, multi-platform support, standardised communication (REST, GraphQL).
        - **API Endpoints**: Specific URL/URI for a resource/functionality. Uses HTTP methods (GET, POST, PUT, PATCH, DELETE).
    - **ORM (Object-Relational Mapping)**: Allows developers to interact with relational databases using their programming language (e.g., Python objects for Django ORM) **without writing raw SQL**.
        - Converts database rows into objects, tables into classes.
        - Simplifies data manipulation, reduces SQL injection risk, improves code readability.
- **Back-End Process to Manage a Web Request**
    - **1. Client Request**: Browser sends HTTP request.
    - **2. Web Server Role**: Server receives request.
        - For static content, retrieves/sends directly.
        - For dynamic content, forwards to web application framework (e.g., Django) via WSGI.
    - **3. Web Framework Role (Application Logic)**: Framework (e.g., Django) processes dynamic request.
        - Routes request to view/controller.
        - Interacts with objects, libraries, databases.
        - **Django's MVT Architecture Components**:
            - **Models (Data Layer)**: Define database table structure as Python classes; Django's ORM maps to tables.
            - **Views (Business Logic Layer)**: Handle logic, bridge models/templates. Process HTTP requests, interact with models, render templates.
            - **Templates (Presentation Layer)**: Render HTML pages dynamically with data.
            - **Django ORM**: Abstracts database layer, maps tables to Python classes for CRUD.
            - **URLs and Routing**: URL dispatcher maps URLs to views.
            - **Forms**: Handles user input, validation, prevents vulnerabilities.
            - **Middleware**: Functions processing requests/responses (security, session management).
            - **Django Admin**: Built-in interface for managing database data.
            - **Authentication/Authorisation**: Comprehensive user system.
            - **API Integration**: Django REST Framework (DRF) builds/manages APIs.
    - **4. Objects**: Instances of models used to represent/manipulate database records via ORMs.
    - **5. Libraries**: Used in back-end code for specific tasks (data processing, validation, integrations).
    - **6. Databases**: Accessed for data storage/retrieval.
- **Creating Files/Directories & Searching Text with Shell Scripts**

    - **Bash (Bourne Again Shell)**: Suitable scripting language for Unix-based systems (Linux, macOS).
    - **Capabilities**:
        - **File Manipulation/Directory Operations**: `mkdir`, `touch`, `mv`, `cp`, `rm`.
        - **Text Processing Capabilities**: `grep` (search text by patterns), `sed` (substitution), `awk` (processing/reporting).
        - Integration with System Commands.
        - Portability.
    - **Practical Use**: Routinely generate file/folder structures for PWA projects, automate tasks (e.g., starting Node.js server).
- **Web-Based Database and SQL Scripts**
    - **SQL (Structured Query Language)**: Programming language for **managing/manipulating data in relational databases**.
    - **Commands**:
        - **SELECT**: Retrieves data, can specify fields (columns). Example: `SELECT title, artist FROM tracks;`.
        - **GROUP BY**: Groups rows with same values into summaries (e.g., calculate averages).
        - **INSERT INTO**: Adds new records (rows).
        - **UPDATE**: Modifies existing records.
        - **DELETE FROM**: Removes records.
        - **DROP TABLE**: Deletes entire table/database.
    - **Constraints (WHERE keyword)**: Filters records based on conditions.
        - **WHERE**: Filters rows.
        - **AND/OR/NOT**: Combine conditions.
        - **LIKE**: Pattern matching with wildcards (%, _).
        - **IN**: Specifies multiple possible values.
    - **Table Joins**: Combine rows from two or more tables based on related columns.
        - **INNER JOIN**: Returns rows with **matches in both tables**.
        - **LEFT OUTER JOIN**: Returns **all rows from left table**, matching right rows (NULLs if no match).
        - **RIGHT OUTER JOIN**: Returns **all rows from right table**, matching left rows (NULLs if no match).
        - **FULL OUTER JOIN**: Returns **all rows from both tables**, combining matches (NULLs for non-matches).
- **Object-Relational Mapping (ORM) vs. SQL**
    - Both methods for working with databases, distinct purposes.
    - **ORM**: Interacts with relational databases using **programming language objects** (e.g., Python) **without raw SQL**.
    - **Comparison Categories**:
        - **Abstraction**:
            - **ORM**: Abstracts database operations through object models, simplifying interaction.
            - **SQL**: Requires detailed queries, precise control.

        - **Flexibility**:
            - **ORM**: Simplifies database interactions with objects.
            - **SQL**: Offers granular control.
        - **Learning Curve**:
            - **ORM**: Has a learning curve (e.g., Django ORM).
            - **SQL**: Generally straightforward, widely used.
        - **Code Readability/Maintenance**:
            - **ORM**: Integrates database operations within programming language, more readable/consistent.
            - **SQL**: Raw SQL can be complex/hard to maintain when scattered.
        - **Security**:
            - **ORM**: Most ORMs automatically handle input sanitisation, **reducing SQL injection risk**.
            - **SQL**: Requires manual sanitisation to prevent injection attacks.
- **Collaborative Development**

    - **Collaboration between front-end and back-end developers is crucial**.
    - **Work Practices**: Clear communication, regular meetings, feedback loops.
    - **Interconnectivity and Workflow**:
        - **UX to UI**: UX provides wireframes/user flows; UI enhances with visuals.
        - **UI to Front-End**: UI passes prototypes; front-end transforms into interactive code (HTML, CSS, JS).
        - **Version Control (Git)**: Multiple developers work simultaneously using branches, facilitates code reviews.
        - **APIs**: **"Glue"** between front-end and back-end, enabling front-end to request/display data dynamically. Front-end uses HTTP methods to request data. Testing APIs is essential (e.g., Postman).
        - **Cross-functional Collaboration**: Leveraging strengths, sharing knowledge.
        - **Iterative Development**: Build, test, refine features incrementally.
        - **Code Consistency/Integration**: Ensuring common patterns and smooth integration.
- **PWA Design Considerations**

    - PWA combines web platform technologies with platform-specific app experience.
    - **Technical Features**: HTML page, CSS, JavaScript, Web APIs. Optional: web app manifest (installation info), service worker (offline experience).
    - **Design Considerations**: Fast loading, push notifications, offline functionality, responsiveness, engaging UI.
    - **Application of Design/UI/UX Principles**:
        - **User Experience (UX)**: Focuses on **user interaction, functionality, usability, user journey**. UX designers conduct research, create personas, user flows, site structure, wireframes.

- **User Interface (UI)**: Focuses on **visual design** (layout, visual elements, interaction patterns). UI designers create visual prototypes, style guides, high-fidelity mockups.
- **Specific Elements Applied**:
    - **Font**: Readable fonts, consistent typography (UI).
    - **Colour**: Color schemes for appeal, brand alignment (UI).
    - **Audio/Video**: Optimisation for network conditions (data compression, encoding).
    - **Navigation**: Intuitive elements (menus, links, buttons).
    - **Accessibility**: Ensuring usability for individuals with disabilities.
        - W3C's **WCAG** provides framework.
        - Considerations: Alt text for images, perceivable/operable/understandable/robust content across assistive technologies (screen readers).

Automation

- **Machine Learning (ML)**
    - **Definition**: A branch of **artificial intelligence (AI)** that develops **algorithms** and **models** enabling computers to **learn from data** and make predictions/decisions without explicit programming.
    - **How it works (keywords)**:
        - **Data**: Foundation; numbers, text, images, audio.
        - **Algorithms**: Mathematical procedures used to learn from data.
        - **Training phase**: Machine learns from a dataset by finding **patterns and relationships**.
        - **Testing phase**: Model evaluated with **new data** to check generalisation performance.
- **Main Types of Machine Learning**
    - **Supervised learning**: Trained on **labelled dataset** (input + correct output). Predicts output based on input. E.g., predicting house prices.
    - **Unsupervised learning**: Trained on data **without labelled outputs**. Finds **patterns or groupings** on its own. E.g., clustering customers.
    - **Reinforcement learning**: Learns by **interacting with an environment**, receives **rewards or penalties** (feedback), updates model to make **optimal decisions**. E.g., game playing, robotics.
- **Applications of Machine Learning**
    - **Image recognition**: Identifying objects, people, scenes (facial recognition).
    - **Natural Language Processing (NLP)**: Understanding human language (chatbots, translation).
    - **Recommendation systems**: Suggesting movies, products based on preferences.
    - **Fraud detection**: Analysing transactions for suspicious activity.
- **Challenges in Using Machine Learning**
    - **Quality of data**: Success heavily depends on quality and quantity of training data.
    - **Overfitting**: Model learns training data too well (including noise), performs poorly on new data.
    - **Bias**: Biased/unrepresentative training data can perpetuate biases in predictions.
- **DevOps vs. MLOps**
    - **DevOps**: Fosters collaboration to create better systems. Focuses on **software development life cycle**.
    - **MLOps**: Focuses on the **ML life cycle**.
    - **Complexity**: ML models are often more complex, requiring specialised tools.
    - **Data**: ML models rely on data for training/inference, introducing data management challenges.

    - **Regulation**: ML models may be subject to regulatory requirements.
- **Artificial Intelligence (AI) vs. Machine Learning (ML)**
    - **AI**:
        - **Broader concept** of machines simulating human intelligence.
        - Goal: Make smart computer systems to solve complex problems like humans.
        - Includes techniques like expert systems, rule-based programming.
        - Can make decisions with limited/no human intervention.
        - Can include systems that don't learn or adapt over time.
        - Applications: Siri, chatbots, intelligent robots.
    - **ML**:
        - **Subset of AI**.
        - Goal: Allow machines to **learn from data** automatically to give accurate output.
        - Always requires **large amounts of data** to function.
        - Primary goal: Enable systems to learn from experience.
        - Focuses on algorithms that **improve through data**.
        - Applications: Recommender systems, Google search algorithms, Facebook auto friend tagging.
- **Variables in Machine Learning**
    - **Variable**: A **characteristic, property, or feature** that changes and represents data.
    - **Features (Independent Input Variables)**: Used by the model to make predictions. E.g., house size, number of bedrooms.
    - **Target Variable (Dependent Output Variable)**: The outcome the model predicts. E.g., house price, Pass/Fail.
- **Traditional Computing vs. Machine Learning**
    - **Traditional Computing**:
        - Purpose: Decision support based on **predefined rules**.
        - Implementation: **Static, rule-based systems**.
        - Adaptability: **Does not adapt** to new information.
    - **Machine Learning**:
        - Purpose: **Predictive modelling** based on **learned patterns**.
        - Implementation: **Dynamic, data-driven learning**.
        - Adaptability: **Learns and improves with new data**.
- **Decision Trees**
    - **Definition**: A **flowchart-like structure** for making decisions based on data, consisting of **nodes** (features), **branches** (decision rules), and **leaves** (outcomes).
    - **How they work**: Algorithm selects features to **split data** into classes, repeating until conditions met. Used for predictions by traversing from root to leaf.
    - **Why software engineers use them**:
        - **Easy to understand and interpret** (visual structure).

        - Works with **numerical and categorical data**.
        - Can identify **important features**.
- **Neural Networks (Artificial Neural Networks - ANNs)**
    - **Definition**: Computational model inspired by biological brains, with **layers of interconnected nodes** (neurons) that process information and learn complex patterns.
    - **Components**:
        - **Node layers**: Input, **hidden**, and output layers.
        - **Nodes (artificial neurons)**: Each is a linear regression model.
        - **Weights**: Numerical values representing **influence** of input on output.
        - **Bias/Threshold**: Part of each node.
        - **Feed forward network**: Data passes from one layer to the next.
    - **How they work**:
        - **Training Cycle**:
            - **Loss function**: Quantifies prediction error.
            - **Backpropagation**: Adjusts network weights to minimise error.
            - **Gradient descent**: Adjusts weights and biases to reduce errors.
        - **Execution Cycle**: Uses trained weights to make predictions on new data via **forward propagation**.
    - **Limitations**:
        - Requires **large amount of data**.
        - **Resource-intensive** training (computational power).
        - Often considered **'black boxes'** (difficult to interpret internal workings).
    - **Types**: Convolutional Neural Networks (CNNs) for image recognition, Recurrent Neural Networks (RNNs) for time series data.
- **Linear Regression**
    - **Purpose**: Predicts a **continuous numerical value** based on continuous input variables.
    - **How it works**: Finds the **best-fitting line** that minimises overall error across all data points.
    - **Use cases**: Predicting house prices, sales forecasting, patient outcomes, student performance, air quality, customer lifetime value.
- **Logistic Regression**
    - **Purpose**: Generates predictions for **categorical data** (e.g., colour, gender).
    - Ideal for **binary classification problems** (e.g., 'cat' or 'not a cat', app usage yes/no, password cracked yes/no).
    - **Data Representation**: Categorical features represented as **binary values (0 or 1)**.
    - **How it works**: Uses the **sigmoid function** to map input features to a **probability between 0 and 1**.
- **K-Nearest Neighbour (KNN)**
    - **Type**: **Supervised learning algorithm**.

    - **Non-parametric**: Makes no assumptions about data distribution.
    - **How it works**: For a new data point, it finds the **'k' closest points (neighbours)** and assigns the new point to the group that most of its neighbours belong to. Primarily used for **classification**.
- **Polynomial Regression**

    - **Purpose**: Used when the relationship between variables is **non-linear**.
    - **Difference from Linear Regression**: Linear assumes a straight line; polynomial **allows for curves** by adding powers of the independent variable.
    - **Degree of the polynomial**: Highest power of 'x' in the equation, determines **flexibility of the curve**. Higher degree can fit data more closely but risks **overfitting**.
    - **How it works**: Input features are **transformed into polynomial features** (e.g., x^2, x^3), then a regression model is fitted.
    - **Use cases**: Car performance (speed vs. fuel), house prices (size vs. price), temperature changes, sales over time, human growth patterns.
- **Bias in Machine Learning**

    - **Human Bias**: Subjective judgements made by individuals during ML/AI development.
        - **Types**:
            - **Selection bias**: Developers choose datasets reflecting their preferences, leading to poor generalisation (e.g., facial recognition on lighter skin).
            - **Confirmation bias**: Prioritising results that confirm existing beliefs.
            - **Cognitive bias**: Subjective judgements influencing feature selection, model design, evaluation.
            - **Implicit bias**: Unconscious biases influencing data (e.g., educators' grading).
        - **Effects**: Inaccurate predictions, perpetuating societal inequalities (e.g., biased hiring algorithms).
    - **Dataset Source Bias**: Data used for training is **unrepresentative** of the target population or real-world scenarios.
        - **Types**:
            - **Sampling bias**: Data collected from specific demographic, not generalisable (e.g., urban health data).
            - **Labelling bias**: Inconsistent or biased labelling criteria by annotators.
            - **Historical bias**: Datasets reflecting past inequalities (e.g., criminal justice data).
            - **Measurement bias**: Inaccuracies/inconsistencies in the data (e.g., grading errors).
            - **Feature bias**: Model relying too heavily on certain features that don't capture full abilities.
        - **Effects**: Disadvantaging specific groups, eroding trust in technology.
        - **Feedback loop**: Predictions influencing future data, reinforcing biases (e.g., student grades).

- **Mitigation Strategies for Bias**
    - Use **explainable AI** techniques.
    - Provide **training** for developers on recognising/mitigating bias.
    - Encourage **diverse teams** in AI development.
    - Conduct **regular audits** and test models on different demographic groups.
    - Implement **fairness metrics**.